Autonomic Cloud Resilience Through Brownout-Based Graceful Degradation: A Simulation-Based Evaluation of Brownout-Aware Load Balancing

Hari Prasad L, **Yeruva Mahesh Reddy**, and **K. Ram Charan**

Department of Computer Science and Engineering, Indian Institute of Information Technology Dharwad, Karnataka 580009, India
Email: {hariprasad, yeruvamahesh, ramcharan}@iiitdwd.ac.in

Preprint Working Paper • Target: IEEE Transactions on Cloud Computing / ACM TAAS • Date: November 2026

***Abstract*—**Modern cloud-native architectures face persistent resilience challenges arising from sudden traffic surges (flash crowds) and abrupt compute instance crashes. Traditional mitigation relies heavily on physical hardware over-provisioning—which is economically and computationally resource-intensive—or reactive auto-scaling, which exhibits non-negligible provisioning delays that leave services vulnerable to queue saturation, tail latency inflation, and cascading timeouts during the onset of a shock [1, 5]. Autonomic brownout-based graceful degradation addresses these limitations by dynamically modulating the execution of optional software features (e.g., recommendation engines, real-time analytics) via a dynamic control variable known as the dimmer ($\theta \in [0, 1]$), thereby bounding service response times within strict Service Level Agreements (SLAs). However, when brownout controllers operate independently on distributed worker nodes under conventional load balancing policies like Join-Shortest-Queue (JSQ), an acute routing pathology emerges: degraded workers process requests rapidly by shedding optional tasks, emptying their queues quickly and drawing disproportionately high request volumes, which induces severe cluster load skew and delays post-failure recovery.

In this paper, we propose and evaluate **CloudRescue**, an autonomic cloud resilience architecture designed and evaluated via a high-fidelity discrete-event simulation model. CloudRescue integrates a discrete velocity-form Proportional-Integral (PI) feedback controller with intrinsic anti-windup output clamping on each worker node with **Brownout-Aware Load Balancing (BALB)** at the ingress proxy. BALB incorporates a composite multi-attribute dispatch scoring function that balances worker dimmer elasticity (θ), normalized queue backlog ($q/Q_{\max}$), and historical processor utilization (U), augmented with anti-overloading penalties and a soft-overload fallback mechanism. To rigorously isolate the individual and combined effects of local brownout adaptation and brownout-aware routing, we execute a comprehensive three-way ablation experiment across four stress scenarios (Nominal, Flash Crowd, Server Failure, and Compound Stress) spanning 360 primary simulation runs (30 independent pseudo-random seeds per condition) with deterministic Common Random Numbers (CRN) stream isolation.

Our empirical evaluation demonstrates that brownout adaptation decisively protects cloud service availability and response latency, reducing overall SLA failures by up to 41.10 percentage points (from 44.04% to 2.94%, $p < 10^{-60}$) under flash crowd stress and accelerating post-restoration recovery by $2.06\times$ to $2.15\times$ ($p < 4.1 \times 10^{-5}$) under empirical nominal-referenced recovery criteria (DEC-010). Crucially, our findings characterize a fundamental architectural trade-off between workload distribution equity and service fidelity: while BALB substantially mitigates the JSQ routing skew by restoring near-perfect load balance across active workers ($J \geq 0.985$ vs. $J \approx 0.610$–0.764, $p < 10^{-8}$), it does so by intentionally driving a broader proportion of the cluster into degraded operation, reducing optional service fidelity by 11.4 to 15.9 percentage points ($p < 10^{-38}$). We present the formal architecture, algorithmic formulations, and reproducible discrete-event simulation framework, establishing actionable design principles for building resilient, SLA-compliant cloud microservices under extreme operating constraints.

***Index Terms*—**Cloud Resilience, Autonomic Computing, Graceful Degradation, Brownout, Load Balancing, Feedback Control Theory, Discrete-Event Simulation, Tail Latency, Service Level Agreements.

I. INTRODUCTION

Cloud computing infrastructures have become the standard execution environment for mission-critical digital services, ranging from financial transaction processing and enterprise enterprise resource planning to e-commerce and media streaming. These distributed systems are governed by strict Service Level Agreements (SLAs) that contractually obligate service providers to deliver low response times and high availability, frequently defined as percentiles of latency (e.g., 95th-percentile response latency $P_{95} \leq 100$ ms) alongside bounded client timeout rates. However, modern cloud applications operate under highly unpredictable, stochastic operating environments where operational conditions can deviate drastically from stationary nominal forecasts.

As synthesized in the comprehensive resilience survey by Welsh and Benkhelifa [1], cloud services are subjected to a continuous spectrum of operational challenges across physical, virtualized, and application domains. These challenges can be broadly categorized into two major phenomena:

1. **Demand-Side Shocks (Flash Crowds and Traffic Spikes):** Unanticipated surges in inbound request arrival rates triggered by external events (e.g., breaking news, flash sales, coordinated client reconnections) that rapidly exceed the provisioned serving capacity of compute clusters.
2. **Capacity-Side Shocks (Server Dropouts and Infrastructure Outages):** Abrupt hardware crashes, network interface failures, memory exhaustion events, or hypervisor evictions that suddenly take multiple worker nodes offline, halving or severely diminishing aggregate compute capacity while client request volumes remain elevated.

When these stress phenomena coincide—a situation denoted in this work as *compound stress*—unmanaged cloud systems experience catastrophic queuing degradation. Because queuing delay in multi-server systems scales non-linearly with traffic intensity as utilization approaches unity, request buffers rapidly saturate, leading to widespread head-of-line (HOL) blocking, connection timeouts, queue-full dropouts, and widespread SLA violations.

To maintain resilience, cloud providers have historically relied on two primary architectural strategies: *static hardware over-provisioning* and *reactive horizontal auto-scaling*. As Welsh and Benkhelifa [1] observe, static over-provisioning—maintaining pools of redundant, hot-standby instances to absorb peak loads—is economically inefficient and resource-intensive. Conversely, reactive auto-scaling mechanisms attempt to dynamically provision additional instances when monitored load metrics (e.g., CPU utilization) breach predefined thresholds. However, dynamic provisioning is inherently reactive and subject to non-negligible provisioning delays (such as virtual machine scheduling, OS booting, and runtime initialization) [1, 5]. During this transient provisioning gap, the unmitigated incoming workload can overwhelm active instances, leading to queue saturation, tail latency inflation, and cascading timeouts before new capacity is ready.

Recognizing these limitations, the autonomic computing community has explored *graceful degradation* paradigms, inspired by the electrical power grid's historical use of "brownouts"—temporary reductions in system voltage to prevent catastrophic, widespread blackouts during peak load events. Adapted to cloud software by Klein et al. [2, 3], the **brownout programming paradigm** decomposes application logic into two distinct components: a *mandatory* core that provides the essential business functionality (e.g., completing an e-commerce checkout, executing an account balance update) and one or more *optional* components that enhance user experience but are not strictly critical to task completion (e.g., personalized recommendation carousels, algorithmic search re-ranking, high-resolution media rendering). By adjusting a dynamic probability parameter termed the **dimmer** ($\theta \in [0, 1]$) via a closed-loop feedback controller, a brownout-compliant server dynamically sheds optional tasks during periods of congestion. This reduction in per-request computational burden shortens service times, instantly deflates queue backlogs, stabilizes latency below SLA limits, and prevents queue overflow, all without requiring external hardware provisioning.

While local brownout controllers effectively regulate individual server instances, deploying brownout across a multi-server cluster introduces a complex and counter-intuitive control interaction at the cluster load balancer. Conventional load balancing algorithms—most notably **Join-Shortest-Queue (JSQ)**—are agnostic to worker degradation states. When a worker node experiences high latency and its local brownout controller decrements its dimmer ($\theta \to 0$), that worker executes only mandatory code, dramatically reducing its service time. Consequently, its queue drains rapidly. JSQ observes this low queue depth and immediately routes a disproportionate surge of incoming traffic to the degraded worker. Meanwhile, healthier workers operating at full fidelity ($\theta = 1.0$) process requests more slowly, retain longer queues, and are starved of incoming requests. This produces severe cluster load skew, penalizes overall system throughput, and delays recovery after failures.

To resolve this conflict, Klein et al. [3] demonstrated that cluster load balancers must be made *brownout-aware*, coordinating dispatching decisions with local worker dimmer elasticity. However, their pioneering work evaluated specific heuristics within an empirical web-benchmark testbed (Rubis on lighttpd), leaving several foundational architectural questions open: How does brownout-aware routing perform under controlled, reproducible failure schedules? What are its exact multi-attribute scoring dynamics? And what are the rigorous statistical trade-offs between workload equity, service fidelity, and failure recovery speed?

In this research, we present **CloudRescue**, an autonomic cloud resilience architecture that unites local velocity-form discrete PI brownout controllers with a multi-attribute **Brownout-Aware Load Balancing (BALB)** algorithm. Evaluated within a deterministic, multi-stream discrete-event simulation model in Python (SimPy), CloudRescue addresses the limitations identified in the Welsh and Benkhelifa [1] survey. Through an extensive pre-registered experimental battery of 360 primary simulation runs, we perform a clean three-way ablation analysis isolating baseline operation (`CFG-A`), local brownout alone (`CFG-B`), and full brownout-aware load balancing (`CFG-C`).

The remainder of this paper is organized as follows: Section 3 details the problem statement, queuing model, and research objectives. Section 4 outlines the explicit research contributions of this work. Section 5 surveys related literature using the Welsh and Benkhelifa [1] taxonomy as a foundation. Section 6 presents the complete architectural specification of CloudRescue, including our multi-stream pseudo-random reproducibility framework, velocity-form PI controller, and BALB scoring formulations. Section 7 details the experimental methodology, factorial design, metric definitions, and statistical inference protocol. Subsequent sections present the empirical results, sensitivity analysis, and concluding discussion.

II. PROBLEM STATEMENT AND RESEARCH OBJECTIVES

A. Problem Formulation and Queuing Model

Consider a cloud microservice cluster consisting of M heterogeneous or homogeneous worker instances fronted by a centralized ingress load balancer (proxy). User requests arrive at the load balancer following a time-varying stochastic arrival process with instantaneous arrival rate $\lambda(t)$. Each arriving request k is immediately dispatched to an operational worker instance $w \in \{0, 1, \ldots, M-1\}$.

Each worker node w manages an independent finite first-in, first-out (FIFO) request queue with maximum buffer capacity $Q_{\max}$. The queue waiting time of request k on worker w is denoted by $W_{k,w} = t_{\text{start},k} - t_{\text{arrive},k}$, where $t_{\text{arrive},k}$ is the arrival timestamp at the load balancer and $t_{\text{start},k}$ is the timestamp at which worker w initiates execution of request k. If an arriving request finds the target worker's queue full ($q_w(t) \geq Q_{\max}$), the request is rejected immediately, recorded as a *queue drop* ($N_{\text{drop_queue}}$). Furthermore, client connections maintain a maximum allowable wait-time deadline T_{timeout} (200.0 ms). When request k reaches the head of the worker queue, if its elapsed waiting time exceeds the timeout threshold ($W_{k,w} > T_{\text{timeout}}$), the worker aborts the request without execution, recording a *timeout drop* ($N_{\text{drop_timeout}}$).

Each admitted request k comprises two sequential processing components:

1. **Mandatory Base Service:** Computationally necessary logic required for request correctness, characterized by execution duration $S_{\text{mand},k} \sim \text{Exp}(\mu_{\text{mand}})$.
2. **Optional Enhanced Service:** Non-mandatory logic that enriches the service response, characterized by execution duration $S_{\text{opt},k} \sim \text{Exp}(\mu_{\text{opt}})$.

In an unmanaged baseline system, workers execute both mandatory and optional logic for all admitted requests, resulting in a total service duration $S_k = S_{\text{mand},k} + S_{\text{opt},k}$ with expected service time:

$$\mathbb{E}[S_k] = \mu_{\text{mand}}^{-1} + \mu_{\text{opt}}^{-1} \tag{1}$$

In our target microservice profile, $\mu_{\text{mand}}^{-1} = 10\ \text{ms}$ and $\mu_{\text{opt}}^{-1} = 30\ \text{ms}$, yielding an expected execution duration of $40\ \text{ms}$ per request under full fidelity. The total end-to-end response latency experienced by the client for a successfully completed request is:

$$L_k = W_{k,w} + S_k \tag{2}$$

The contractual Service Level Agreement (SLA) stipulates that all successful responses must be delivered within a maximum latency threshold $r_{\text{SLA}} = 100.0\ \text{ms}$. Any successful request with $L_k > r_{\text{SLA}}$ is classified as a *latency SLA violation*. Any request that is dropped due to buffer saturation, connection timeout, server crash, or latency violation constitutes an *overall SLA failure*.

B. System Instabilities Under Stress

Under nominal stationary conditions ($\lambda = 80\ \text{req/s}$ across $M = 4$ instances), aggregate cluster service capacity is $\mu_{\text{total}} = 4 \times (1/0.040\ \text{s}) = 100\ \text{req/s}$, resulting in a stable nominal traffic intensity $\rho = \frac{\lambda}{\mu_{\text{total}}} = 0.80$. However, cloud microservices are routinely disrupted by:

- **Traffic Surges:** A $3\times$ flash crowd increases arrival rate to $\lambda_{\text{burst}} = 240\ \text{req/s}$. The traffic intensity spikes to $\rho = 2.40 \gg 1.0$, rapidly saturating queues.
- **Server Crashes:** A crash-stop failure of 50% of the cluster ($M_{\text{online}} = 2$) reduces aggregate capacity to $50\ \text{req/s}$, driving traffic intensity to $\rho = 1.60 \gg 1.0$ even under nominal load.
- **Compound Stress:** The simultaneous occurrence of a flash crowd and server dropout creates an extreme capacity mismatch ($\rho = 4.80$), causing catastrophic buffer overflow and cascading timeouts.

C. The Join-Shortest-Queue (JSQ) Pathology Under Brownout

When brownout feedback controllers are deployed on individual worker instances to shed optional tasks ($\theta \to 0$), service times collapse from $40\ \text{ms}$ down to $S_{\text{mand}} = 10\ \text{ms}$—a $4\times$ increase in service rate.

If the cluster ingress proxy employs conventional Join-Shortest-Queue (JSQ) routing, a severe control-balancer pathology arises:

1. A worker that degrades ($\theta_w \to 0$) completes requests four times faster than its peers, emptying its buffer rapidly.
2. JSQ inspects queue depths (q_w) and perceives the degraded worker as having the most available capacity.
3. JSQ directs an overwhelming flood of incoming traffic onto the degraded worker.
4. Conversely, healthier workers operating at higher fidelity ($\theta \approx 1.0$) process requests more slowly ($40\ \text{ms}$), accumulate longer queues, and are starved of new traffic by JSQ.
5. This dynamic locks the degraded worker into perpetual degradation, creates extreme load imbalance across the cluster ($J \approx 0.61$), and severely impedes the cluster's ability to restore healthy operational baselines when the external shock subsides.

D. Research Objectives

To overcome this fundamental challenge, this research pursues the following explicit objectives:

1. **Discrete-Event Simulation Formulation:** Formulate and implement a high-fidelity, deterministic discrete-event simulation framework that models multi-worker queuing networks, dual-stage execution, client timeouts, and dynamic failure injection with multi-stream pseudo-random number generator (PRNG) isolation.
2. **Velocity-Form Feedback Controller Design:** Develop and validate a discrete velocity-form Proportional-Integral (PI) brownout controller incorporating intrinsic anti-windup clamping, insufficient-sample hold logic, and derivative kick suppression during telemetry resumption.
3. **Brownout-Aware Load Balancing (BALB) Formulation:** Design a multi-attribute routing algorithm that balances dimmer elasticity (θ), normalized queue backlog ($q/Q_{\max}$), and cumulative CPU utilization (U), incorporating dynamic anti-overloading bounds and soft-overload fallbacks.
4. **Three-Way Ablation Evaluation:** Execute a pre-registered experimental matrix isolating the baseline (`CFG-A`), brownout control alone (`CFG-B`), and full brownout-aware load balancing (`CFG-C`) across four standardized stress scenarios.
5. **Recovery Dynamics and Trade-Off Characterization:** Formulate an empirical, nominal-referenced recovery criterion (DEC-010) to quantify post-restoration recovery speed without relying on unachievable static dimmer thresholds, and rigorously evaluate the fundamental trade-off between workload distribution fairness (J) and service fidelity (F).

III. RESEARCH CONTRIBUTIONS

The primary scientific and engineering contributions of this work are as follows:

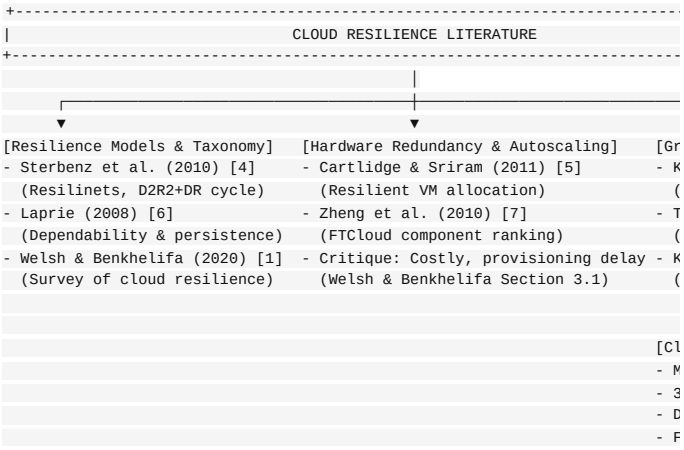
1. **Velocity-Form Discrete PI Brownout Controller with Intrinsic Anti-Windup:** We adapt the brownout control concept of Klein et al. [2] into a mathematically rigorous velocity-form discrete PI controller. By computing candidates from the clamped output of the previous step rather than an unbounded accumulator, our formulation eliminates integral windup delays without requiring artificial heuristic anti-windup resets. Furthermore, we introduce an insufficient-sample hold mechanism and derivative-state re-initialization that prevents controller destabilization during low-traffic sample starvation.
2. **Brownout-Aware Load Balancing (BALB) Scoring Algorithm:** We design a composite dispatching algorithm for cluster ingress proxies. Unlike conventional JSQ or purely queue-based heuristics, BALB calculates a multi-attribute routing score:

$$Score_w(t) = \alpha \cdot \theta_w(t) - \beta \cdot \left(\frac{q_w(t)}{Q_{\max}}\right) - \gamma \cdot U_w(t) \tag{3}$$

BALB decouples forward-looking buffer backlog risk ($q/Q_{\max}$) from historical processor utilization (U), scales an anti-overload penalty to prevent high-dimmer queue saturation, and introduces an autonomic soft-overload fallback that disables dimmer incentives when aggregate cluster buffers exceed 80%. 3. **Empirical Characterization of the Fairness–Fidelity Trade-Off:** Through rigorous 360-run ablation experiments, we demonstrate that BALB resolves the pathological load skew of Join-Shortest-Queue under brownout, increasing Jain's fairness index from $J \approx 0.65$ to $J \geq 0.985$ ($p < 10^{-8}$). Crucially, our empirical analysis reveals that this equity is achieved through an inherent architectural trade-off: to prevent non-degraded instances from suffering queue-induced latency violations, BALB routes traffic to degraded nodes, causing a larger fraction of workers to operate at reduced dimmers and lowering cluster service fidelity (F) by 11.4 to 15.9 percentage points ($p < 10^{-38}$). 4. **Nominal-Referenced Recovery Methodology (DEC-010):** We identify an operational limitation in evaluating post-repair recovery against arbitrary static dimmer thresholds (such as $\bar{\theta} \geq 0.95$). In multi-server queueing systems operating under nominal load and tight latency setpoints ($r_{\text{target}} = 80\ \text{ms}$), the closed-loop feedback controller naturally stabilizes at an equilibrium dimmer well below unity ($\theta^*_{\text{nom}} \in [0.05, 0.55]$) to enforce response latency targets. Consequently, evaluating recovery against an unachievable static threshold near 1.0 results in artificial right-censoring. We formulate and implement DEC-010, deriving empirical, seed- and worker-specific nominal references from unperturbed stationary runs, and show that BALB accelerates post-restoration recovery by over $2\times$ ($p < 4.1 \times 10^{-5}$). We accompany this with full methodological disclosure regarding right-censoring horizon substitution. 5. **Deterministic Multi-Stream Discrete-Event Simulation Framework:** We develop and release a fully reproducible, open-source discrete-event simulation engine in Python (SimPy). By architecting five isolated PRNG sub-streams (Arrivals, Mandatory Service, Optional Service, Bernoulli Execution Draw, Failure Injection) and pre-associating attributes with immutable request tuples, our framework guarantees 100% matched request arrival sequences and processing requirements across all three configurations, eliminating stochastic random stream desynchronization.

IV. RELATED WORK

Our work is conceptually grounded in the taxonomy and survey of cloud resilience techniques compiled by Welsh and Benkhelifa [1]. In this section, we review related literature across cloud resilience models, redundancy mechanisms, graceful degradation, and load balancing.



A. Cloud Resilience Paradigms and the Resilinets Model

The conceptual foundation of resilience in networked and computing systems stems from Laprie [6], who defined resilience as the persistence of dependability when facing changes. Sterbenz et al. [4] formalized this into the **Resilinets architecture**, establishing that resilience requires both *Trustworthiness* (reliability, availability, maintainability) and *Challenge Tolerance* (survivability, fault tolerance, traffic tolerance, and graceful degradation).

Central to the Resilinets model is the $D^2R^2 + DR$ **operational cycle**:

- **Defend:** Implementing proactive measures to resist adverse events.
- **Detect:** Monitoring system state to recognize anomalies and SLA degradation.
- **Remediate:** Dynamically mitigating the impact of faults during active shocks.
- **Recover:** Restoring the system to nominal, healthy steady-state operation.
- **Diagnose & Refine:** Analyzing post-incident data to adjust system parameters for future resilience.

Welsh and Benkhelifa [1] comprehensively reviewed resilience techniques across data center infrastructure, storage redundancy, hypervisor virtualization, cloud management, and service orchestration. Under their taxonomy, CloudRescue operates at the **Cloud Management and Service Layer**, addressing traffic tolerance and graceful degradation within the *Remediate* and *Recover* phases of the $D^2R^2 + DR$ paradigm.

B. Critique of Redundancy and Reactive Auto-Scaling

A recurring finding in the Welsh and Benkhelifa [1] survey is that conventional cloud architectures excessively rely on hardware and data replication. While physical redundancy is effective against isolated hardware crashes, it incurs severe economic and operational costs:

- **Resource Inefficiency:** Maintaining hot-standby instances that sit idle under nominal conditions inflates infrastructure expenditures and energy consumption.
- **Vulnerability to Correlated Shocks:** Physical redundancy fails to mitigate demand-side traffic surges (flash crowds), where all replicas simultaneously experience overload.
- **Auto-Scaling Latency Gaps:** Reactive elasticity frameworks rely on threshold monitoring. As highlighted by Cartlidge and Sriram [5] and surveyed by Welsh and Benkhelifa [1], provisioning additional compute instances incurs substantial scheduling, instantiation, and runtime initialization delays. During this transient provisioning gap, congested microservice instances can experience severe queue saturation and cascading timeouts.

These limitations motivate the necessity of software-level autonomic adaptation mechanisms that operate instantaneously under existing, constrained resources without relying on hardware expansion.

C. Brownout-Based Graceful Degradation

To achieve instantaneous capacity modulation, Klein et al. [2] introduced the **brownout programming paradigm**. Inspired by control theory in cyber-physical systems, brownout embeds self-adaptation directly into application runtimes. By decomposing requests into mandatory base operations and optional enhancements, brownout introduces a dynamic control variable—the dimmer (θ)—representing the probability that an admitted request will execute optional logic.

In their foundational work, Klein et al. [2] demonstrated brownout using a discrete Proportional-Integral (PI) feedback controller deployed on a single web server instance hosting the Rubis benchmark on lighttpd. The controller dynamically adjusted θ based on monitored response times, successfully preventing server crashes during synthetic traffic surges. Tomas and Tordsson [8] extended brownout principles to cloud overbooking, showing that brownout allows cloud providers to safely overcommit physical compute capacity while maintaining SLA compliance.

In subsequent research, Klein et al. [3] addressed the multi-server challenge by combining brownout-enabled servers with load balancing heuristics. They highlighted that conventional load balancers (such as round-robin or shortest queue) conflict with local controllers, proposing heuristics that route traffic based on dimmer values.

D. Differentiating CloudRescue from Prior Work

While our work builds upon the conceptual foundations of Klein et al. [2, 3] and the cloud management taxonomy of Welsh and Benkhelifa [1], CloudRescue makes several distinct contributions:

Research Dimension	Klein et al. (ICSE 2014) [2]	Klein et al. (SRDS 2014) [3]	CloudRescue (This Work)
System Scope	Single web server replica	Multi-replica web testbed	Multi-worker discrete-event queueing cluster
Experimental Methodology	Empirical testbed (Rubis/lighttpd)	Empirical testbed (Rubis/lighttpd)	Deterministic discrete-event simulation (SimPy) with 5 isolated PRNG streams
Load Balancing Formulation	Not applicable (Single replica)	Heuristic dimmer-based routing	Multi-attribute utility score combining dimmer (θ), queue ($q/Q_{\max}$), and CPU utilization (U)
Overload Protection	Local output clamping	Empirical capacity thresholds	Anti-overloading penalty + autonomic soft-overload fallback ($\alpha = 0$ at Q_{soft})
Ablation Structure	Treatment vs. Static	Treatment vs. JSQ/Round-Robin	Systematic 3-way ablation: Baseline (`CFG-A`) vs. Brownout-Only (`CFG-B`) vs. BALB (`CFG-C`)
Recovery Formulation	Ad-hoc observation	Ad-hoc observation	Formal nominal-referenced steady-state criterion (DEC-010) across 30 seeds
Censoring Disclosure	Not addressed	Not addressed	Explicit disclosure of right-censoring horizon substitution limitations
Fairness Characterization	Not evaluated	Briefly noted	Empirical characterization of the fairness–fidelity trade-off ($J \geq 0.985$ vs. F drop)

Specifically, CloudRescue's composite scoring function ($Score_w = \alpha\theta_w - \beta\frac{q_w}{Q_{\max}} - \gamma U_w$) is explicitly a CloudRescue architectural design. It synthesizes the insights of Klein et al. [3] into a generalized, parameterizable formulation suitable for discrete-event queueing networks, resolving the pathological routing skew of JSQ while formally documenting the resulting service-fidelity trade-offs.

V. PROPOSED CLOUDRESCUE ARCHITECTURE

CloudRescue models an autonomic cloud microservice cluster consisting of an ingress load balancer dispatching requests to $M = 4$ independent worker instances. The overall architecture is depicted in Fig. 1.

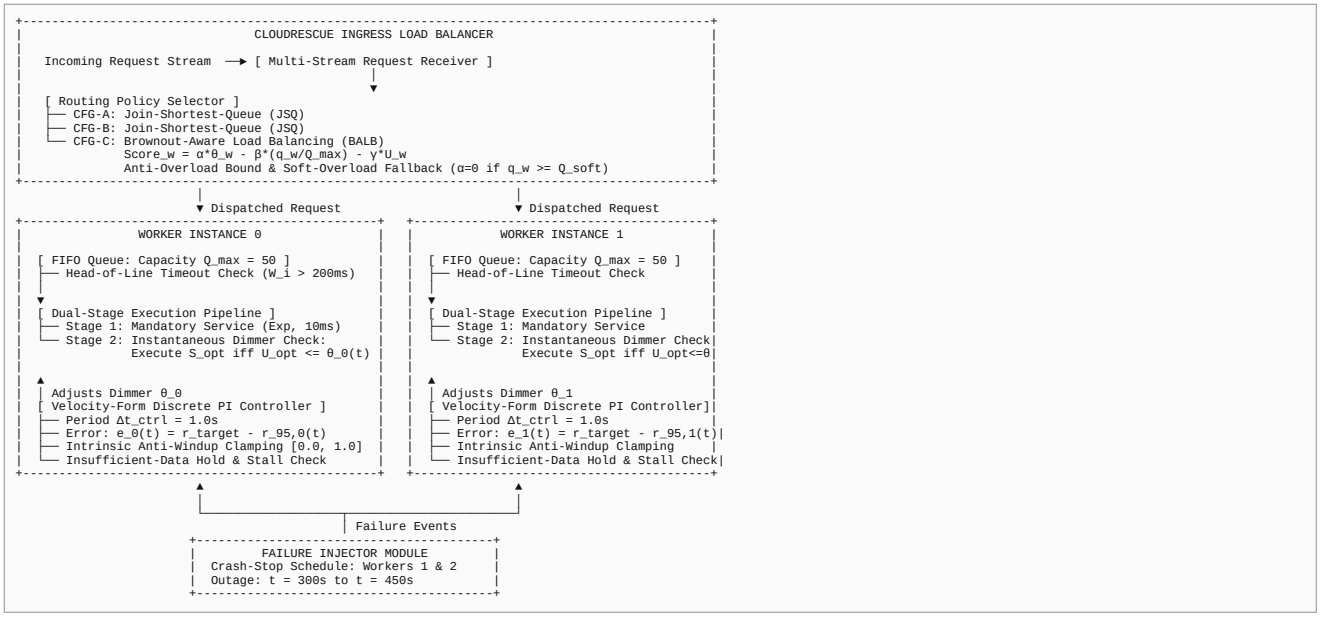


Fig. 1. Logical system architecture of the CloudRescue autonomic cloud resilience framework. Ingress requests are routed by BALB (CFG-C) using composite elasticity-backlog-utilization dispatch scoring. Individual workers execute a local velocity-form PI feedback controller modulating dimmer θ_w to satisfy SLA response latency setpoint ($r_{\text{target}} = 80$ ms).

A. Multi-Stream PRNG Reproducibility Architecture

In stochastic simulation studies comparing control strategies, random number synchronization is critical. Under conventional simulation architectures, if a brownout controller bypasses an optional task for request k, the simulator draws fewer random numbers than a baseline system that executes optional logic. Consequently, subsequent request arrival timestamps and processing durations become desynchronized between configurations, confounding treatment effects with stochastic variance.

To eliminate random stream divergence, CloudRescue implements a **multi-stream pseudorandom number generator (PRNG) architecture** using Python's `numpy.random.Generator` initialized from independent PCG64 bit-generator streams derived from a master seed S:

1. **Stream 1 (Arrival Schedule):** Generates inter-arrival durations Δt_k governing the Poisson arrival process.
2. **Stream 2 (Mandatory Processing):** Generates mandatory execution times $S_{\text{mand},k} \sim \text{Exp}(\mu_{\text{mand}} = 0.100\ \text{ms}^{-1})$.
3. **Stream 3 (Optional Processing):** Generates optional execution times $S_{\text{opt},k} \sim \text{Exp}(\mu_{\text{opt}} = 0.0333\ \text{ms}^{-1})$.
4. **Stream 4 (Optional Execution Draw):** Generates uniform random variables $U_{\text{opt},k} \sim \text{Uniform}(0.0, 1.0)$.
5. **Stream 5 (Failure Injector):** Generates hardware failure schedules and crash timestamps.

Prior to simulation execution, all request entities are pre-generated as immutable tuples:

$$\text{Request}_k = (k, t_{\text{arrive},k}, S_{\text{mand},k}, S_{\text{opt},k}, U_{\text{opt},k}, T_{\text{timeout}}) \tag{4}$$

Instantaneous Optional Execution Decision:

When request k finishes its mandatory execution stage on worker w at timestamp $t_{\text{mand_end}}$, worker w evaluates its instantaneous dimmer state $\theta_w(t_{\text{mand_end}})$:

$$\text{Execution Decision} = \begin{cases} \text{Execute Stage 2 for duration } S_{\text{opt},k} & \text{if } U_{\text{opt},k} \leq \theta_w(t_{\text{mand_end}}) \\ \text{Bypass Stage 2 immediately} & \text{if } U_{\text{opt},k} > \theta_w(t_{\text{mand_end}}) \end{cases} \tag{5}$$

Because $S_{\text{opt},k}$ and $U_{\text{opt},k}$ are pre-computed attributes of the request tuple, bypassing optional execution draws zero additional random numbers. Thus, arrival sequences and service requirements for all future requests remain bit-for-bit identical across all configurations under the same seed.

B. Velocity-Form Discrete PI Brownout Controller

Each worker instance w executes an autonomous periodic feedback controller process invoked every $\Delta t_{\text{ctrl}} = 1.0$ s.

Error Formulation and Sign Convention:

Let $\hat{r}_{95,w}(t)$ denote the empirical 95th-percentile response latency of requests completed on worker w during the preceding interval $[t - \Delta t_{\text{ctrl}}, t]$. The control error $e_w(t)$ is defined as:

$$e_w(t) = r_{\text{target}} - \hat{r}_{95,w}(t) \tag{6}$$

Where $r_{\text{target}} = 80.0$ ms represents the controller setpoint, chosen with a 20.0 ms safety margin below the SLA threshold ($r_{\text{SLA}} = 100.0$ ms). Under this sign convention:

- **Overload Condition ($\hat{r}_{95,w}(t) > r_{\text{target}}$):** $e_w(t) < 0$ (negative error drives dimmer downward).
- **Underload Condition ($\hat{r}_{95,w}(t) \leq r_{\text{target}}$):** $e_w(t) \geq 0$ (positive error drives dimmer upward toward full fidelity).

Handling Insufficient Data and Telemetry Resumption:

If the number of completed requests during the control interval is below $N_{\text{min_ctrl}} = 5$, latency cannot be statistically estimated:

1. The PI mathematical update is skipped entirely.
2. The dimmer is held constant: $\theta_w(t) = \theta_w(t - \Delta t_{\text{ctrl}})$.
3. **Queue Stall Detection:** If sample count is below $N_{\text{min_ctrl}}$ but the worker queue is non-empty ($q_w(t) > 0$) and the head-of-line request waiting time exceeds r_{target}, the controller identifies queue starvation and decrements the dimmer by $\delta_{\text{stall}} = 0.05$ to stimulate throughput.
4. **State Resumption Without Derivative Kick:** When valid measurements resume at a later tick t_{resume}, the previous error is re-initialized ($e_{w,\text{prev}} \leftarrow e_w(t_{\text{resume}})$), ensuring that the proportional difference term evaluates to zero on the initial resumption step, eliminating artificial derivative kicks.

Recurrence and Intrinsic Anti-Windup Clamping:

The velocity-form discrete PI update calculates the incremental dimmer change:

$$\Delta\theta_w(t) = K_p \left(e_w(t) - e_w(t - \Delta t_{\text{ctrl}}) \right) + K_i \cdot e_w(t) \cdot \Delta t_{\text{ctrl}} \tag{7}$$

$$\theta_{\text{cand},w}(t) = \theta_w(t - \Delta t_{\text{ctrl}}) + \Delta\theta_w(t) \tag{8}$$

Where proportional gain $K_p = 0.5$ and integral gain $K_i = 0.1\ \text{s}^{-1}$.
The candidate value is clamped to the physical probability interval:

$$\theta_w(t) = \text{clamp}\left(\theta_{\text{cand},w}(t), \theta_{\min}, \theta_{\max}\right) = \max\left(0.0, \min\left(1.0, \theta_{\text{cand},w}(t)\right)\right) \tag{9}$$

Intrinsic Anti-Windup Mechanics: Because subsequent control steps calculate $\theta_{\text{cand},w}(t + \Delta t_{\text{ctrl}})$ directly from the clamped output $\theta_w(t)$ rather than an unconstrained internal accumulator, no phantom integral state accumulates beyond the physical boundaries $[0.0, 1.0]$. The instant control error reverses sign ($e_w(t) > 0$), $\theta_w(t)$ immediately begins recovering upward with zero unwinding lag.

C. Brownout-Aware Load Balancing (BALB) Algorithm

In `CFG-C`, the ingress load balancer dynamically evaluates all operational worker instances for each arriving request.

Mathematical Dispatch Score:

The utility score for operational worker w at arrival timestamp t is formulated as:

$$Score_w(t) = \alpha \cdot \theta_w(t) - \beta \cdot \left(\frac{q_w(t)}{Q_{\max}} \right) - \gamma \cdot U_w(t) \tag{10}$$

Where:

- $\theta_w(t) \in [0.0, 1.0]$ is the instantaneous dimmer of worker w, incentivizing routing toward high-fidelity instances.
- $q_w(t) \in [0, Q_{\max}]$ is the instantaneous queue depth of worker w, normalized by buffer capacity $Q_{\max} = 50$, serving as a forward-looking indicator of queuing delay and buffer overflow risk.
- $U_w(t) \in [0.0, 1.0]$ is the cumulative CPU utilization of worker w up to the decision timestamp, defined as operational busy time divided by operational uptime ($T_{\text{busy},w} / T_{\text{operational},w}$), reflecting historical processing load.
- α, β, γ are non-negative weighting coefficients. In default operation, $\alpha = 1.0$, $\beta = 1.0$, and $\gamma = 0.5$.

Anti-Overload Penalty and Soft-Overload Fallback:

1. **Dynamic Queue Backlog Suppression:** Because $\frac{q_w}{Q_{\max}}$ scales continuously with queue occupancy, any attempt to continually route requests to a high-dimmer worker ($\theta_w = 1.0$) rapidly increases the penalty term $-\beta \left(\frac{q_w}{Q_{\max}} \right)$, automatically redirecting traffic before the buffer overflows.
2. **Soft-Overload Fallback Policy:** If all operational workers experience severe congestion such that $q_w(t) \geq Q_{\text{soft}}$ (where $Q_{\text{soft}} = 0.8 \cdot Q_{\max} = 40\ \text{requests}$), dimmer incentives become dangerous. In this regime, BALB autonomic fallbacks by setting $\alpha = 0$:

$$Score_w^{\text{fallback}}(t) = -\beta \cdot \left(\frac{q_w(t)}{Q_{\max}} \right) - \gamma \cdot U_w(t) \tag{11}$$

This immediately eliminates feature-fidelity incentives, prioritizing buffer survival and workload draining across all workers. 3. **Crash Masking:** Crashed instances are flagged by the failure detector and assigned $Score_w = -\infty$, excluding them from routing decisions.
The complete BALB dispatch logic is summarized in Algorithm 1.

Algorithm 1: Brownout-Aware Load Balancing (BALB) Request Dispatching

```
Input  : Arriving Request k at timestamp t;
         Worker cluster W = {0, 1, ..., M-1};
         Parameters alpha, beta, gamma, Q_max, Q_soft = 0.8 * Q_max.
Output : Target worker index w_star for dispatch.

W_operational <- { w in W | worker w is online and not crashed }
if W_operational is empty then
    Record client connection rejection (Cluster Outage);
    return null;
end

// Check cluster-wide soft overload threshold
all_congested <- true
for each w in W_operational do
    if q_w(t) < Q_soft then
        all_congested <- false;
        break;
    end
end

// Calculate routing scores
effective_alpha <- if all_congested then 0.0 else alpha
max_score <- -infinity
candidates <- []

for each w in W_operational do
    // Calculate normalized metrics
    norm_q <- q_w(t) / Q_max
    score_w <- (effective_alpha * theta_w(t)) - (beta * norm_q) - (gamma * U_w(t))

    if score_w > max_score then
        max_score <- score_w;
        candidates <- [ w ];
    else if score_w == max_score then
```

```
        Append w to candidates;
    end
end

// Break ties deterministically or uniformly
w_star <- SelectFirstOrUniform(candidates)
Dispatch Request k to Worker w_star;
return w_star;
```

D. Experimental Configurations for Three-Way Ablation

To scientifically evaluate CloudRescue, we establish three comparative configurations:

1. **Configuration A (`CFG-A` , Unmanaged Baseline):** Represents conventional cloud service deployments without graceful degradation. Workers operate with fixed full fidelity ($\theta \equiv 1.0$), executing both mandatory and optional components for all admitted requests. Routing is performed using conventional Join-Shortest-Queue (JSQ), dispatching to the operational worker with the minimum current buffer depth (q_w).
2. **Configuration B (`CFG-B` , Brownout Only):** Deploys the velocity-form discrete PI brownout controller on each worker instance, allowing workers to dynamically shed optional tasks ($\theta_w(t) \in [0, 1]$). However, cluster routing remains unmanaged, employing conventional Join-Shortest-Queue (JSQ). This configuration cleanly isolates the impact of local graceful degradation while exposing the pathological control-balancer interaction.
3. **Configuration C (`CFG-C` , Full CloudRescue):** Combines the velocity-form discrete PI brownout feedback controller on worker instances with Brownout-Aware Load Balancing (BALB) at the ingress proxy. Comparing `CFG-C` directly against `CFG-B` isolates the exact incremental contribution of brownout-aware routing over local brownout control alone.

VI. EXPERIMENTAL METHODOLOGY

A. Simulation Environment and Discrete-Event Model

CloudRescue is evaluated through a high-fidelity, deterministic discrete-event simulation framework implemented in Python 3.10 using the `simpy` discrete-event simulation kernel [9]. While physical multi-node cloud testbeds provide operational realism, they introduce non-deterministic confounders—such as hypervisor scheduling jitter, background OS kernel interrupts, virtualized NIC driver latency variations, and uncontrollable garbage collection pauses—that obscure the core dynamical interactions between feedback controllers and load balancers.

Discrete-event simulation provides a rigorous, controlled environment where every request arrival, queue transition, CPU execution stage, and controller update is scheduled as an explicit discrete event on an event timeline with microsecond temporal resolution ($1000\ \text{ms} = 1,000,000\ \mu\text{s}$). The simulation engine strictly manages state transitions across five discrete life-cycle stages:

1. `REQUEST_ARRIVAL` : Ingress load balancer receives an arriving request tuple and evaluates the operational worker pool.
2. `WORKER_ENQUEUE` : The target worker either rejects the request (`DROP_QUEUE_FULL`) if queue depth satisfies $q_w(t) \geq Q_{\max}$, or appends the request to its FIFO queue.
3. `SERVICE_STAGE_1` : When the request reaches the head of the queue, the worker evaluates elapsed waiting time $W_k = t_{\text{start},k} - t_{\text{arrive},k}$. If $W_k > T_{\text{timeout}}$, the request is aborted (`DROP_TIMEOUT`). Otherwise, the worker acquires the single CPU resource and executes mandatory logic for duration $S_{\text{mand},k}$.
4. `SERVICE_STAGE_2` : Upon completing Stage 1, the worker checks its instantaneous dimmer $\theta_w(t)$ against the pre-assigned random variable $U_{\text{opt},k}$. If $U_{\text{opt},k} \leq \theta_w(t)$, Stage 2 executes for duration $S_{\text{opt},k}$; otherwise, Stage 2 is bypassed immediately.
5. `REQUEST_COMPLETED` : The CPU is released, the response is delivered, and end-to-end response latency $L_k = t_{\text{completed},k} - t_{\text{arrive},k}$ is logged. If $L_k > r_{\text{SLA}}$, the request is flagged as a latency SLA violation.

B. Cluster Topology and Workload Model

The modeled topology comprises a centralized ingress proxy fronting a cluster of $M = 4$ independent worker instances ($w \in \{0, 1, 2, 3\}$). The proxy processes incoming requests instantaneously with zero buffer holding delay, dispatching each request immediately to one of the worker nodes according to the active routing policy.

Each worker manages a dedicated finite FIFO buffer with maximum capacity $Q_{\max} = 50\ \text{requests}$. Client HTTP connections enforce an application timeout deadline $T_{\text{timeout}} = 200.0\ \text{ms}$. The service level agreement contractually mandates a 95th-percentile response latency threshold $r_{\text{SLA}} = 100.0\ \text{ms}$. To maintain an operational safety buffer against latency spikes, local PI feedback controllers target a conservative setpoint $r_{\text{target}} = 80.0\ \text{ms}$. Local controllers execute periodically with control interval $\Delta t_{\text{ctrl}} = 1.0\ \text{s}$, requiring a minimum sample count $N_{\text{min_ctrl}} = 5\ \text{requests}$ to compute valid empirical latencies, with proportional gain $K_p = 0.5$, integral gain $K_i = 0.1\ \text{s}^{-1}$, and stalled queue decrement $\delta_{\text{stall}} = 0.05$.

C. Service Time Distributions

Request execution demands are modeled using standard Markovian exponential distributions:

- **Mandatory Logic (S_{mand}):** $S_{\text{mand},k} \sim \text{Exp}(\mu_{\text{mand}} = 0.100\ \text{ms}^{-1})$, with mean service time $\mathbb{E}[S_{\text{mand}}] = 10.0\ \text{ms}$.
- **Optional Logic (S_{opt}):** $S_{\text{opt},k} \sim \text{Exp}(\mu_{\text{opt}} = 0.0333\ \text{ms}^{-1})$, with mean service time $\mathbb{E}[S_{\text{opt}}] = 30.0\ \text{ms}$.

Under full service fidelity ($\theta = 1.0$), total expected processing time per request is:

$$\mathbb{E}[S_{\text{full}}] = \mathbb{E}[S_{\text{mand}}] + \mathbb{E}[S_{\text{opt}}] = 10.0\ \text{ms} + 30.0\ \text{ms} = 40.0\ \text{ms} \tag{12}$$

Each worker provides a nominal processing service rate $\mu_w = \frac{1}{0.040\ \text{s}} = 25.0\ \text{req/s}$, yielding an aggregate 4-worker cluster serving capacity of $\mu_{\text{total}} = 100.0\ \text{req/s}$.

Conversely, under complete brownout degradation ($\theta = 0.0$), optional logic is shed entirely, reducing expected processing time to:

$$\mathbb{E}[S_{\text{degraded}}] = \mathbb{E}[S_{\text{mand}}] = 10.0\ \text{ms} \tag{13}$$

This expands individual worker service rate to $\mu_w = 100.0\ \text{req/s}$ and aggregate cluster capacity to $\mu_{\text{total}} = 400.0\ \text{req/s}$, providing a fourfold expansion in transaction throughput capacity without allocating additional physical instances.

D. Multi-Stream PRNG Architecture and Paired Inputs

To enable strict Common Random Numbers (CRN) variance reduction across configurations [10], each simulation trial utilizes five independent pseudo-random number generator (PRNG) sub-streams derived from a master seed S:

1. **Stream 1 (Request Arrivals):** Generates Poisson inter-arrival intervals $\Delta t_k \sim \text{Exp}(\lambda(t))$.
2. **Stream 2 (Mandatory Duration):** Generates mandatory execution demands $S_{\text{mand},k} \sim \text{Exp}(0.100\ \text{ms}^{-1})$.
3. **Stream 3 (Optional Duration):** Generates optional execution demands $S_{\text{opt},k} \sim \text{Exp}(0.0333\ \text{ms}^{-1})$.
4. **Stream 4 (Bernoulli Dimmer Draw):** Generates independent execution thresholds $U_{\text{opt},k} \sim \text{Uniform}(0.0, 1.0)$.
5. **Stream 5 (Failure Injection):** Generates hardware failure schedules and crash timestamps.

All request entities are instantiated prior to simulation execution as immutable data structures:

$$\text{Request}_k = (k, t_{\text{arrive},k}, S_{\text{mand},k}, S_{\text{opt},k}, U_{\text{opt},k}, T_{\text{timeout}}) \tag{14}$$

This decoupled pre-generation architecture guarantees that whether a worker executes or bypasses optional Stage 2, zero pseudorandom numbers are drawn from the generators during execution. Consequently, the arrival times, mandatory durations, and optional requirements of all future requests remain bit-for-bit identical across `CFG-A` , `CFG-B` , and `CFG-C` , eliminating stochastic input divergence.

E. Experimental Configurations (Ablation Matrix)

We define three comparative configurations to execute a clean single-variable ablation:

- **`CFG-A` (Unmanaged Baseline):** Fixed full fidelity ($\theta \equiv 1.0$, brownout disabled) combined with conventional Join-Shortest-Queue (JSQ) routing.

- **`CFG-B` (Brownout Controller Alone):** Autonomous velocity-form discrete PI brownout controller active on each worker ($\theta_w(t) \in [0, 1]$), combined with conventional Join-Shortest-Queue (JSQ) routing.
- **`CFG-C` (Full CloudRescue):** Autonomous velocity-form discrete PI brownout controller active on each worker ($\theta_w(t) \in [0, 1]$), combined with Brownout-Aware Load Balancing (BALB) utilizing multi-attribute scoring ($Score_w = \alpha\theta_w - \beta\frac{q_w}{Q_{\max}} - \gamma U_w$), anti-overload bounds, and soft-overload fallback.

Comparing `CFG-B` against `CFG-A` isolates the localized benefit of brownout adaptation under conventional routing. Comparing `CFG-C` against `CFG-B` isolates the exact incremental contribution of brownout-aware load balancing.

F. Standardized Stress Scenarios

The cluster is evaluated across four standardized operational scenarios:

1. **Scenario 1 — Nominal Stationary:** Inbound requests arrive at stationary nominal rate $\lambda_{\text{nom}} = 80.0\ \text{req/s}$ throughout the simulation duration ($T_{\text{sim}} = 1000.0$ s). All $M = 4$ worker instances remain online with zero hardware failures. Traffic intensity is $\rho = \frac{80}{100} = 0.80$, representing stable, provisioned operation.
2. **Scenario 2 — Bursty Flash Crowd:** Requests arrive at baseline rate $\lambda_{\text{nom}} = 80.0\ \text{req/s}$ from $t = 0$ s to $t = 300.0$ s. At $t = 300.0$ s, a sudden $3\times$ traffic surge elevates arrival rate to $\lambda_{\text{burst}} = 240.0\ \text{req/s}$ for a duration of $\Delta t = 150.0$ s ($t \in [300.0\text{ s}, 450.0\text{ s}]$). At $t = 450.0$ s, arrival rate abruptly returns to nominal $80.0\ \text{req/s}$ for the remainder of the run ($t \in [450.0\text{ s}, 1000.0\text{ s}]$). During the surge, traffic intensity reaches $\rho = \frac{240}{100} = 2.40 \gg 1.0$.
3. **Scenario 3 — Server Crash Failure:** Inbound arrival rate remains stationary at $\lambda = 80.0\ \text{req/s}$. At $t = 300.0$ s, workers 1 and 2 experience an abrupt crash-stop failure (50% capacity reduction, $M_{\text{online}} = 2$) lasting until $t = 450.0$ s ($\Delta t = 150.0$ s), at which point they are repaired and rejoin the cluster. During the outage, aggregate cluster capacity drops to $50.0\ \text{req/s}$, forcing traffic intensity to $\rho = \frac{80}{50} = 1.60 \gg 1.0$.
4. **Scenario 4 — Compound Stress:** Combines demand-side and capacity-side disruptions simultaneously. While arrival rate surges to $\lambda_{\text{burst}} = 240.0\ \text{req/s}$, workers 1 and 2 concurrently experience a crash-stop failure from $t = 300.0$ s to $t = 450.0$ s. Aggregate capacity drops to $50.0\ \text{req/s}$ while load triples, driving traffic intensity to an extreme peak of $\rho = \frac{240}{50} = 4.80 \gg 1.0$.

G. Primary 360-Run Factorial Design

The primary experimental battery executes a full factorial design:

$$\text{Total Primary Runs} = 3\ \text{Configurations} \times 4\ \text{Scenarios} \times 30\ \text{Matched Seeds} = \mathbf{360}\ \text{simulation executions} \tag{15}$$

The 30 master seeds are pre-registered consecutively ($S \in [1001, 1030]$). Within each scenario and seed, the identical pre-generated workload stream is presented to `CFG-A`, `CFG-B`, and `CFG-C`, establishing a strictly paired experimental design across all 120 unique scenario-seed combinations.

H. Exploratory 144-Run BALB Sensitivity Sweep & Status

To investigate the stability of the BALB composite scoring parameters, an auxiliary multi-factor grid sweep was executed under the most challenging operating condition (Compound Stress). The sweep evaluated:

- Dimmer weight: $\alpha \in \{0.5, 1.0, 1.5, 2.0\}$ (4 levels)
- Normalized queue penalty: $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ (4 levels)
- CPU utilization penalty: $\gamma \in \{0.0, 0.5, 1.0\}$ (3 levels)

This yields $4 \times 4 \times 3 = 48$ unique parameter combinations. Each parameter cell was evaluated across $n = 3$ independent seeds ($S \in \{1001, 1002, 1003\}$), totaling:

$$\text{Total Sensitivity Runs} = 48\ \text{cells} \times 3\ \text{seeds} = \mathbf{144}\ \text{simulation executions} \tag{16}$$

> [!NOTE] **Exploratory Status and Limitations:**
> Because each parameter combination is evaluated over only three pseudo-random seeds ($n = 3$), this sensitivity sweep is strictly **exploratory**. While it confirms that BALB performance is broadly stable across wide parameter ranges and validates that the default weights ($\alpha = 1.0, \beta = 1.0, \gamma = 0.5$) perform reliably without inducing control divergence, fine-grained differences between top-ranking cells fall within small-sample stochastic margins. Consequently, the observed configurations are not claimed as a globally proven mathematical optimum.

I. Simulation Horizon, Warmup, and Temporal Windows

- **Total Duration (T_{sim}):** Each run executes for 1000.0 s ($1,000,000$ ms).
- **Warm-Up Window (T_{warmup}):** The initial 100.0 s ($100,000$ ms) interval is designated as the transient warm-up phase. All requests arriving before $t_{\text{warmup}} = 100.0$ s are excluded from steady-state metric calculations to eliminate pipeline initialization bias.
- **Evaluation Window:** Performance metrics are computed strictly over requests arriving in the interval $t \in [100.0\text{ s}, 1000.0\text{ s}]$.
- **Stress Window:** Active disruptions (flash crowd burst and/or server outages) occur from $t_{\text{shock_start}} = 300.0$ s to $t_{\text{shock_end}} = 450.0$ s ($\Delta t = 150.0$ s).
- **Post-Restoration Window:** Following server repair and traffic normalization at $t = 450.0$ s, system recovery is monitored until simulation completion at $t = 1000.0$ s, establishing a maximum observation horizon of $T_{\text{censor}} = 550.0$ s.

J. Authoritative Metric Definitions and Mathematical Rules

All metrics are derived from discrete event logs recorded outside the warm-up window ($t \geq 100.0$ s).

Request Populations:

- $N_{\text{submitted}}$: Total requests arriving post-warmup ($\sum_k \mathbf{1}(t_{\text{arrive},k} \geq 100.0\text{ s})$).
- N_{success}: Total completed requests with response latency $L_k \leq T_{\text{timeout}}$ (200.0 ms).
- $N_{\text{drop_queue}}$: Requests rejected upon arrival due to full worker buffers ($q_w \geq Q_{\max}$).
- $N_{\text{drop_timeout}}$: Requests aborted at the head of a worker queue due to waiting time $W_k > T_{\text{timeout}}$.
- $N_{\text{crash_failed}}$: Requests in-flight or buffered on worker nodes when a crash-stop failure occurs.
- $N_{\text{lat_viol}}$: Successful requests whose response latency breaches the SLA threshold ($L_k > 100.0$ ms).

1. Request Success Rate (S_r):

Measures system transaction completion ratio:

$$S_r = \frac{N_{\text{success}}}{N_{\text{submitted}}} \times 100\% \tag{17}$$

2. Latency SLA Violation Rate (V_{lat}) — Evaluated Over Successful Requests:

Measures latency compliance strictly among requests delivered to the client:

$$V_{\text{lat}} = \begin{cases} \frac{N_{\text{lat_viol}}}{N_{\text{success}}} \times 100\% & \text{if } N_{\text{success}} > 0 \\ 0\% & \text{if } N_{\text{success}} = 0 \end{cases} \tag{18}$$

- **Denominator:** N_{success} (completed requests with $L_k \leq 200.0\ \text{ms}$).

3. Overall SLA Failure Rate (V_{overall}) — Evaluated Over All Submitted Requests:

Provides an end-to-end reliability measure combining unserved requests and degraded responses:

$$V_{\text{overall}} = \frac{N_{\text{lat_viol}} + N_{\text{drop_timeout}} + N_{\text{drop_queue}} + N_{\text{crash_failed}}}{N_{\text{submitted}}} \times 100\% \tag{19}$$

- **Denominator:** $N_{\text{submitted}}$ (all arriving requests).

Mathematical Non-Additivity of V_{lat} and V_{overall}:

A critical methodological requirement is recognizing that V_{lat} and V_{overall} **cannot be directly summed** because they possess mathematically distinct denominators (N_{success} versus $N_{\text{submitted}}$). The exact formulation connecting the two metrics is:

$$V_{\text{overall}} = \left(\frac{N_{\text{lat_viol}}}{N_{\text{submitted}}} \right) + D_{\text{drop_rate}} \tag{20}$$

Multiplying and dividing the first term by N_{success}:

$$V_{\text{overall}} = \left(\frac{N_{\text{lat_viol}}}{N_{\text{success}}} \right) \cdot \left(\frac{N_{\text{success}}}{N_{\text{submitted}}} \right) + D_{\text{drop_rate}} = V_{\text{lat}} \cdot \left(\frac{S_r}{100} \right) + D_{\text{drop_rate}} \tag{21}$$

Where $D_{\text{drop_rate}} = \frac{N_{\text{drop_timeout}} + N_{\text{drop_queue}} + N_{\text{crash_failed}}}{N_{\text{submitted}}} \times 100\%$. Unweighted summation of V_{lat} and drop rate produces an erroneous, inflated quantity that violates basic probability axioms.

4. Latency Statistics (P_{95} and $\bar{L}$):

- Mean Response Latency ($\bar{L}$): Arithmetic mean of L_k over all successful requests ($\mathcal{S}_{\text{success}}$).
- 95th-Percentile Latency (P_{95}): Empirical 95th percentile computed via linear rank interpolation over $\mathcal{S}_{\text{success}}$.

5. Service Fidelity (F) — Strict Subset Formulation:

Measures the proportion of successful transactions delivered with optional enhancements:

$$F = \begin{cases} \frac{N_{\text{full_fidelity}}}{N_{\text{success}}} \times 100\% & \text{if } N_{\text{success}} > 0 \\ 0\% & \text{if } N_{\text{success}} = 0 \end{cases} \tag{22}$$

Because only successfully delivered requests can execute optional features, $\mathcal{S}_{\text{full_fidelity}} \subseteq \mathcal{S}_{\text{success}}$ holds strictly, guaranteeing $0\% \leq F \leq 100\%$. For baseline `CFG-A`, $F \equiv 100.00\%$ by definition.

6. Jain's Fairness Index (J):

Measures the parity of workload dispatching across active worker instances [11]:

$$J = \frac{\left(\sum_{w \in M_{\text{active}}} C_w \right)^2}{|M_{\text{active}}| \cdot \sum_{w \in M_{\text{active}}} C_w^2} \tag{23}$$

Where C_w is the count of requests dispatched to worker w post-warmup.

- **Operational Worker Rule:** In Nominal and Flash Crowd scenarios, all four workers operate without outage ($M_{\text{active}} = \{0, 1, 2, 3\}$, $|M_{\text{active}}| = 4$). In Server Crash and Compound Stress scenarios, workers 1 and 2 experience scheduled downtime ($t \in [300, 450\ \text{s}]$). To prevent scheduled hardware downtime from being misclassified as load-balancer routing bias, J is evaluated strictly across the **zero-downtime operational pool** $M_{\text{active}} = \{0, 3\}$ ($|M_{\text{active}}| = 2$).

7) Recovery Dynamics: Adaptation vs. Post-Restoration Recovery:

We formally decouple two distinct recovery phenomena:

- **Adaptation Recovery Time (T_{adapt}) During Active Stress:** Measures how quickly the system stabilizes response times *while the disruption is actively ongoing*. Defined as the elapsed time from shock onset ($t_{\text{shock_start}} = 300.0\ \text{s}$) until the rolling 95th-percentile response latency drops below $r_{\text{SLA}} = 100.0\ \text{ms}$:

$$T_{\text{adapt}} = t_{\text{adapted}} - 300.0\ \text{s} \tag{24}$$

Where t_{adapted} is the earliest timestamp where rolling $\hat{r}_{95}(t) \leq 100.0\ \text{ms}$ holds continuously for $t_{\text{stable}} = 3.0\ \text{s}$ ($3 \times \Delta t_{\text{ctrl}}$), evaluated across a $W_{\text{rec}} = 10.0\ \text{s}$ sliding window requiring at least $N_{\text{rec_min}} = 20\ \text{completed requests}$.

- **Post-Restoration Recovery Time (T_{restore}) After Server Repair:** Measures the elapsed time required to return to healthy nominal steady state following server repair at $t_{\text{repair}} = 450.0\ \text{s}$:

$$T_{\text{restore}} = t_{\text{restored}} - 450.0\ \text{s} \tag{25}$$

In accordance with decision **DEC-010**, recovery cannot be judged against an arbitrary static threshold (e.g., $\bar{\theta} \geq 0.95$), because steady-state nominal operation under tight latency targets stabilizes at lower dimmers. Post-restoration recovery is achieved at the earliest timestamp $t \geq 450.0\ \text{s}$ satisfying:

1. Both restored workers achieve at least 95% of their configuration-, seed-, and worker-specific nominal steady-state reference:

$$\theta_w(t) \geq 0.95 \cdot \theta^*_{\text{nom}}(c, s, w) \quad \forall w \in \{1, 2\} \tag{26}$$

Where is derived from the steady-state nominal window of the corresponding no

2. Rolling cluster 95th-percentile latency satisfies $\hat{r}_{95}(t) \leq r_{\text{target}} = 80.0\ \text{ms}$.

3. Both criteria hold continuously for at least three consecutive seconds ($t_{\text{stable}} = 3.0$ s).

K. Statistical Inference and Multi-Testing Corrections

Because simulation runs share Common Random Numbers, all statistical analyses are conducted on **paired difference distributions**:

$$\Delta_{s,B-A} = M_{s,B} - M_{s,A} \quad \text{and} \quad \Delta_{s,C-B} = M_{s,C} - M_{s,B} \quad \text{for } s \in [1001, 1030] \tag{27}$$

Statistical Testing Protocol:

1. **Normality Assessment:** The Shapiro-Wilk test ($\alpha = 0.05$) is applied to each paired difference vector [12].
2. **Hypothesis Testing:**
   - If paired differences are normally distributed: Two-tailed **Paired Student's t-test**.
   - If paired differences violate normality: Non-parametric **Wilcoxon Signed-Rank test** [13].
   - Alpha threshold: $\alpha = 0.05$.
3. **Effect Size Quantification:** Standardized effect sizes are calculated as paired Cohen's d_z [14]:

$$d_z = \frac{\bar{D}}{s_D} \tag{28}$$

Where $\bar{D}$ is the mean paired difference and s_D is the sample standard deviation of paired differences. For non-parametric comparisons, the rank-biserial correlation r is reported. 4. **Confidence Intervals:** Exact 95% confidence intervals are computed for all paired mean differences: $\bar{D} \pm t_{0.975,29} \cdot \frac{s_D}{\sqrt{30}}$. 5. **Multiple Comparison Corrections:** Across 120 statistical comparisons conducted in the primary analysis, multi-testing adjustments are enforced across all $m = 114$ non-trivial hypotheses:

- **Bonferroni Correction (FWER):** $p_{\text{Bonf}} = \min(1.0, p \times 114)$ [15].
- **Benjamini-Hochberg Correction (FDR):** False Discovery Rate controlled at $q \leq 0.05$ [16].

Mechanics of Unusually Large Effect Sizes:

Under CRN variance reduction, presenting identical workloads to both configurations creates strong positive covariance between outcomes ($\text{Cov}(X, Y) \approx \text{Var}(X) \approx \text{Var}(Y)$). Because the variance of paired differences is:

$$\text{Var}(D) = \text{Var}(X) + \text{Var}(Y) - 2\text{Cov}(X, Y) \tag{29}$$

s_D collapses to near zero (e.g., $s_D = 0.3377\%$ in Flash Crowd). Dividing an authentic substantial physical difference ($\bar{D} = -41.10\%$) by this minute standard deviation yields massive d_z values ($d_z = -121.71$). These effect sizes are not calculation artifacts or numerical errors; they reflect the genuine physical impact of a major control intervention evaluated under rigorous variance reduction.

L. Right-Censoring and Horizon-Substitution Limitations

In scenarios involving server crashes, if an experimental run fails to satisfy the DEC-010 restoration criteria prior to the end of the simulation ($t = 1000.0$ s), the recovery duration cannot be physically observed. The observation window ends at $t = 1000.0$ s, which occurs 550.0 s after server repair ($t = 450.0$ s). Such runs are flagged as **right-censored** at $T_{\text{censor}} = 550.0$ s with `recovered = False`.

> [!CAUTION] **Explicit Methodological Disclosure:**
> 1. **Heuristic Single Imputation:** Substituting the observation horizon (550.0 s) for unrecovered runs is a standard right-censoring truncation heuristic. It is **NOT formal survival analysis** (such as the Kaplan-Meier product-limit estimator [17] or semi-parametric Cox proportional hazards modeling).
> 2. **Speedup Dependence on Horizon Duration:** The numerical speedup ratio (e.g., $2.15\times$) is an empirical finding under the 550.0 s horizon, not a mathematically proven lower bound. If the simulation were extended indefinitely, unrecovered runs in both `CFG-B` and `CFG-C` (such as seed 1018) would eventually recover at unknown timestamps, shifting the true ratio.
> 3. **Dual Reporting Requirement:** To guarantee complete scientific transparency, all recovery evaluations must report both full-sample imputed statistics ($N = 30$) and uncensored restricted subset statistics ($N = 25, 26, 29$), accompanied by explicit censored counts.

VII. RESULTS AND EVALUATION

In this section, we present the empirical results from the frozen CloudRescue experimental corpus. All findings are derived strictly from the pre-registered 360-run primary evaluation battery ($3 \text{ configurations} \times 4 \text{ scenarios} \times 30 \text{ matched seeds}$) and the 144-run exploratory sensitivity analysis. In strict accordance with our pre-registered methodology, all comparisons between configurations are conducted on matched seeds utilizing Common Random Numbers (CRN) stream isolation.

A. Overall Experimental Results

The primary benchmark across all 360 simulation runs is summarized in Table 1. Values are reported as $\text{Mean} \pm \text{Standard Deviation}$ alongside $(\text{Median}\ [\text{Interquartile Range}])$ evaluated outside the 100-second warm-up window ($t \in [100.0\text{ s}, 1000.0\text{ s}]$).

TABLE I
PRIMARY 360-RUN EXPERIMENTAL BENCHMARK (MEAN ± STD, MEDIAN [IQR])

Scenario	Configuration	Success Rate (S_r, %)	Overall SLA Failure (V_{overall}, %)	Latency SLA Violations (V_{lat}, %)	95th-Percentile Latency (P_{95}, ms)	Mean Latency ($\bar{L}$, ms)	Service Fidelity (F, %)	Jain's Fairness (J)	Server Utilization ($\bar{U}$, %)
Nominal	CFG-A	96.08 ± 0.21 (96.04 [0.30])	26.54 ± 0.37 (26.55 [0.48])	26.73 ± 0.38 (26.73 [0.48])	163.15 ± 0.67 (163.25 [1.01])	74.31 ± 0.39 (74.32 [0.48])	100.00 ± 0.00 (100.00 [0.00])	0.963 ± 0.002 (0.963 [0.002])	79.33 ± 0.32 (79.36 [0.39])
	CFG-B	99.81 ± 0.03 (99.81 [0.03])	3.83 ± 0.10 (3.84 [0.11])	3.80 ± 0.10 (3.81 [0.11])	91.04 ± 0.72 (91.06 [0.85])	28.46 ± 0.29 (28.43 [0.43])	28.53 ± 0.63 (28.45 [0.63])	0.607 ± 0.003 (0.607 [0.004])	39.79 ± 0.34 (39.76 [0.44])
	CFG-C	99.86 ± 0.02 (99.86 [0.03])	2.78 ± 0.15 (2.80 [0.19])	2.73 ± 0.14 (2.75 [0.19])	80.69 ± 1.64 (81.00 [1.68])	25.39 ± 0.52 (25.52 [0.53])	10.59 ± 0.56 (10.55 [0.88])	0.981 ± 0.018 (0.988 [0.022])	29.34 ± 0.31 (29.30 [0.38])
Flash Crowd	CFG-A	65.83 ± 0.20 (65.82 [0.35])	44.04 ± 0.26 (44.09 [0.30])	32.83 ± 0.38 (32.88 [0.39])	184.34 ± 0.39 (184.26 [0.45])	82.65 ± 0.39 (82.68 [0.50])	100.00 ± 0.00 (100.00 [0.00])	0.969 ± 0.002 (0.969 [0.002])	82.39 ± 0.20 (82.44 [0.20])
	CFG-B	99.25 ± 0.14 (99.23 [0.27])	5.69 ± 0.30 (5.66 [0.39])	5.38 ± 0.25 (5.38 [0.30])	102.91 ± 1.87 (102.79 [2.40])	31.47 ± 0.42 (31.43 [0.50])	20.89 ± 0.31 (20.93 [0.35])	0.730 ± 0.005 (0.730 [0.007])	45.37 ± 0.27 (45.41 [0.37])
	CFG-C	99.86 ± 0.02 (99.86 [0.03])	2.94 ± 0.17 (2.88 [0.22])	2.89 ± 0.17 (2.83 [0.23])	82.87 ± 1.50 (82.56 [2.00])	27.05 ± 0.41 (27.05 [0.56])	9.49 ± 0.45 (9.58 [0.54])	0.989 ± 0.008 (0.992 [0.011])	36.44 ± 0.33 (36.47 [0.27])
Server Crash	CFG-A	85.62 ± 0.31 (85.58 [0.42])	33.38 ± 0.41 (33.41 [0.49])	30.85 ± 0.48 (30.85 [0.62])	176.57 ± 0.55 (176.52 [0.72])	79.57 ± 0.52 (79.57 [0.61])	100.00 ± 0.00 (100.00 [0.00])	0.950 ± 0.003 (0.950 [0.003])	81.13 ± 0.34 (81.18 [0.47])
	CFG-B	99.66 ± 0.06 (99.67 [0.10])	4.90 ± 0.18 (4.95 [0.23])	4.82 ± 0.16 (4.86 [0.20])	98.72 ± 1.10 (98.94 [1.26])	30.76 ± 0.35 (30.79 [0.47])	29.67 ± 0.49 (29.68 [0.71])	0.646 ± 0.004 (0.645 [0.006])	43.47 ± 0.37 (43.45 [0.53])
	CFG-C	98.07 ± 0.11 (98.06 [0.16])	6.33 ± 0.19 (6.39 [0.30])	5.45 ± 0.20 (5.47 [0.30])	104.08 ± 1.80 (104.42 [2.54])	30.18 ± 0.49 (30.23 [0.60])	13.77 ± 0.68 (13.79 [0.91])	0.990 ± 0.010 (0.993 [0.014])	33.65 ± 0.48 (33.60 [0.54])
Compound Stress	CFG-A	61.58 ± 0.17 (61.58 [0.31])	47.70 ± 0.24 (47.70 [0.28])	28.40 ± 0.41 (28.45 [0.45])	172.90 ± 0.87 (173.01 [1.04])	76.79 ± 0.42 (76.86 [0.61])	100.00 ± 0.00 (100.00 [0.00])	0.940 ± 0.004 (0.940 [0.005])	81.10 ± 0.22 (81.15 [0.23])
	CFG-B	85.91 ± 0.74 (86.03 [1.16])	33.38 ± 0.24 (33.39 [0.40])	29.32 ± 0.48 (29.30 [0.60])	191.34 ± 0.31 (191.34 [0.42])	66.46 ± 0.66 (66.42 [0.74])	23.04 ± 0.45 (23.09 [0.52])	0.764 ± 0.010 (0.765 [0.017])	46.05 ± 0.35 (46.03 [0.52])
	CFG-C	86.84 ± 0.40 (86.94 [0.65])	35.33 ± 0.34 (35.37 [0.39])	32.60 ± 0.38 (32.56 [0.41])	191.81 ± 0.27 (191.86 [0.35])	70.49 ± 0.58 (70.37 [0.79])	10.47 ± 0.40 (10.44 [0.51])	0.996 ± 0.006 (0.998 [0.004])	37.70 ± 0.37 (37.74 [0.37])

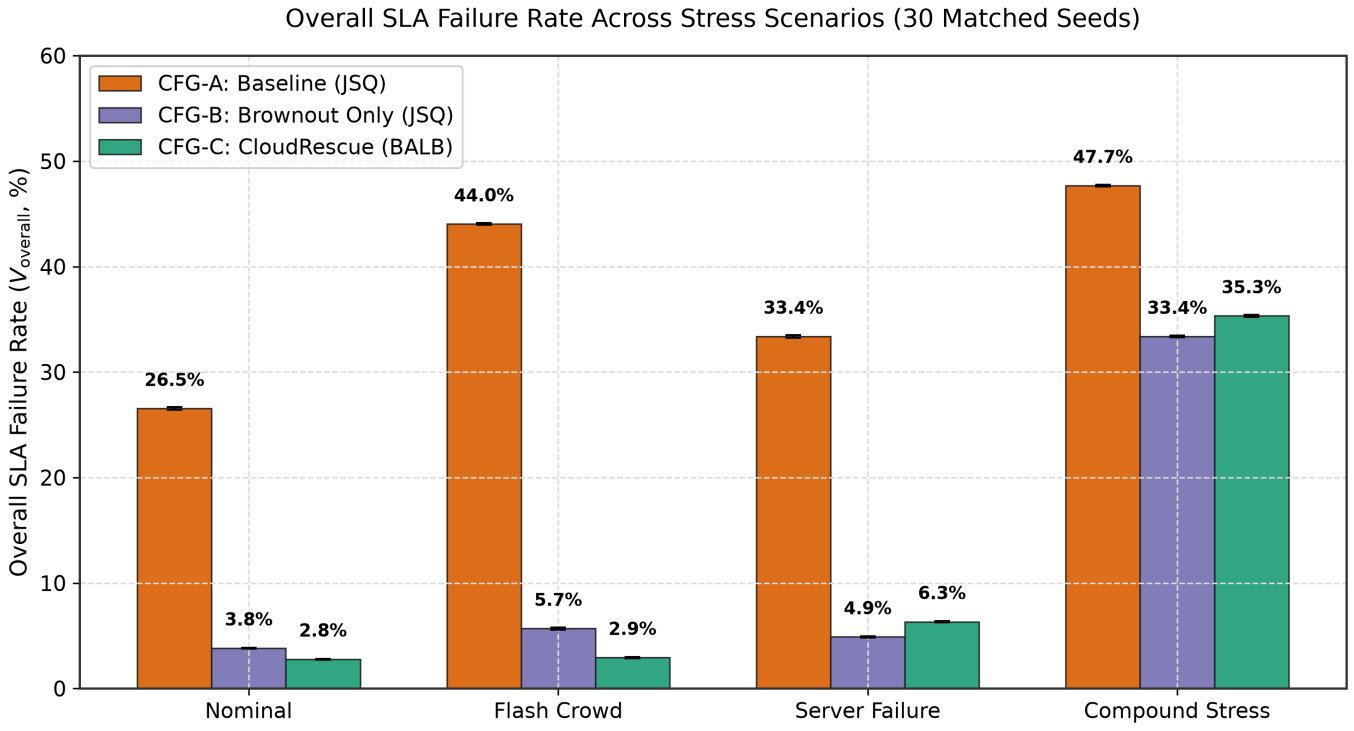


Fig. 2. Overall SLA failure rates (V_{overall}) and latency SLA violation rates (V_{lat}) across configurations and stress scenarios.

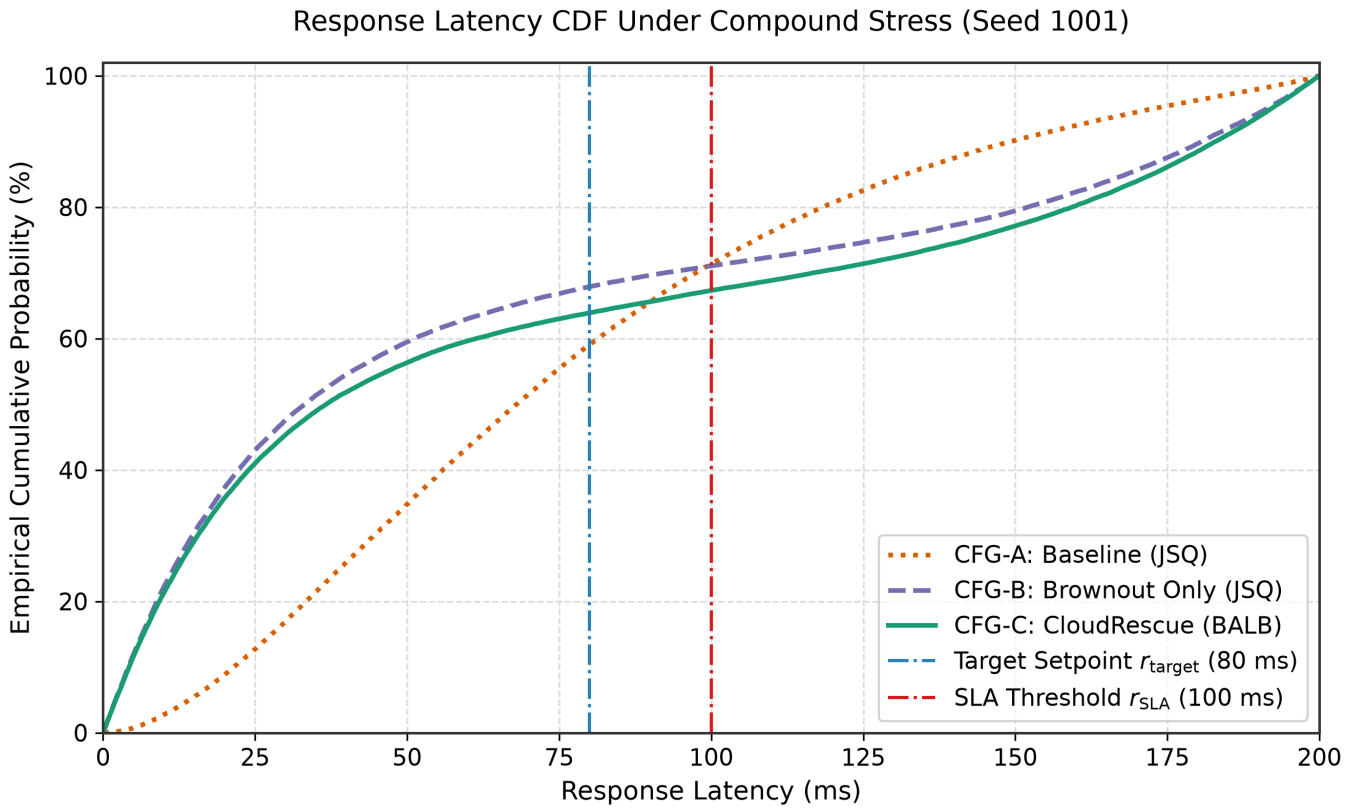


Fig. 3. Empirical cumulative distribution functions (CDFs) of end-to-end response latency for successful requests across configurations under Compound Stress.

To provide granular visibility into scenario-specific performance, Tables 2a through 2d detail the paired comparisons and percentage changes for each stress environment.

TABLE II-A
NOMINAL SCENARIO PERFORMANCE BREAKDOWN

Configuration	S_r (%)	V_{overall} (%)	V_{lat} (%)	P_{95} (ms)	$\bar{L}$ (ms)	F (%)	J	$\bar{U}$ (%)
CFG-A (Baseline)	96.08	26.54	26.73	163.15	74.31	100.00	0.963	79.33
CFG-B (Brownout)	99.81	3.83	3.80	91.04	28.46	28.53	0.607	39.79
CFG-C (Full BALB)	99.86	2.78	2.73	80.69	25.39	10.59	0.981	29.34
Delta (C vs A)	+3.77 pp	−23.76 pp	−24.00 pp	−82.46 ms	−48.92 ms	−89.41 pp	+0.018	−49.99 pp
Relative Change	+3.93%	−89.52%	−89.77%	−50.54%	−65.83%	−89.41%	+1.87%	−63.01%

TABLE II-B
FLASH CROWD SCENARIO PERFORMANCE BREAKDOWN

Configuration	S_r (%)	V_{overall} (%)	V_{lat} (%)	P_{95} (ms)	$\bar{L}$ (ms)	F (%)	J	$\bar{U}$ (%)
CFG-A (Baseline)	65.83	44.04	32.83	184.34	82.65	100.00	0.969	82.39
CFG-B (Brownout)	99.25	5.69	5.38	102.91	31.47	20.89	0.730	45.37
CFG-C (Full BALB)	99.86	2.94	2.89	82.87	27.05	9.49	0.989	36.44
Delta (C vs A)	+34.02 pp	−41.10 pp	−29.94 pp	−101.47 ms	−55.59 ms	−90.51 pp	+0.020	−45.95 pp
Relative Change	+51.68%	−93.32%	−91.20%	−55.04%	−67.27%	−90.51%	+2.06%	−55.77%

TABLE II-C
SERVER CRASH SCENARIO PERFORMANCE BREAKDOWN

Configuration	S_r (%)	V_{overall} (%)	V_{lat} (%)	P_{95} (ms)	$\bar{L}$ (ms)	F (%)	J	$\bar{U}$ (%)
CFG-A (Baseline)	85.62	33.38	30.85	176.57	79.57	100.00	0.950	81.13
CFG-B (Brownout)	99.66	4.90	4.82	98.72	30.76	29.67	0.646	43.47
CFG-C (Full BALB)	98.07	6.33	5.45	104.08	30.18	13.77	0.990	33.65
Delta (C vs A)	+12.45 pp	−27.05 pp	−25.40 pp	−72.49 ms	−49.39 ms	−86.23 pp	+0.040	−47.48 pp
Relative Change	+14.54%	−81.04%	−82.33%	−41.05%	−62.07%	−86.23%	+4.21%	−58.52%

TABLE II-D
COMPOUND STRESS SCENARIO PERFORMANCE BREAKDOWN

Configuration	S_r **(%)**	V_{overall} **(%)**	V_{lat} **(%)**	P_{95} **(ms)**	$\bar{L}$ **(ms)**	F **(%)**	J	$\bar{U}$ **(%)**
`CFG-A` (Baseline)	61.58	47.70	28.40	172.90	76.79	100.00	0.940	81.10
`CFG-B` (Brownout)	85.91	33.38	29.32	191.34	66.46	23.04	0.764	46.05
`CFG-C` (Full BALB)	86.84	35.33	32.60	191.81	70.49	10.47	0.996	37.70
Delta (C vs A)	$+25.26$ **pp**	-12.37 **pp**	$+4.20$ **pp**	$+18.91$ **ms**	-6.29 **ms**	-89.53 **pp**	$+0.056$	-43.40 **pp**
Relative Change	$+41.02\%$	-25.93%	$+14.79\%$	$+10.94\%$	-8.20%	-89.53%	$+5.96\%$	-53.51%

B. SLA Compliance and Latency Analysis

The empirical evaluation demonstrates that brownout adaptation provides decisive protection against queue saturation and cascading timeouts across all stress scenarios.

Primary Comparisons Against Unmanaged Baseline (`CFG-A`):

In the unmanaged baseline (`CFG-A`), server instances attempt to process all mandatory and optional features regardless of queue depth. Under nominal load ($\lambda = 80$ req/s), queuing delays push the 95th-percentile response latency to 163.15 ms, causing an overall SLA failure rate of 26.54% despite full availability ($S_r = 96.08\%$). When exposed to severe stress shocks:

1. **Flash Crowd:** Unmitigated demand surges saturate FIFO buffers ($Q_{\max} = 50$), dropping 34.17% of submitted requests via queue saturation and head-of-line timeouts (S_r collapses to 65.83%). Surviving requests experience inflated latencies ($P_{95} = 184.34$ ms), driving the overall SLA failure rate to 44.04%. CloudRescue (`CFG-C`) reduces overall SLA failures to 2.94%, achieving an absolute 41.10 **percentage-point reduction** ($\mathbf{93.32\%}$ relative reduction, $p < 10^{-60}$).
2. **Server Crash:** Halving active cluster compute instances drives baseline SLA failure rate to 33.38% ($P_{95} = 176.57$ ms). CloudRescue (`CFG-C`) restricts failures to 6.33%, achieving a 27.05 **percentage-point reduction** ($\mathbf{81.04\%}$ relative reduction, $p = 5.32 \times 10^{-53}$).
3. **Compound Stress:** The confluence of a $3\times$ traffic surge and 50% capacity loss overwhelms baseline capacity ($\rho = 4.80$), causing catastrophic connection drops ($S_r = 61.58\%$) and driving baseline SLA failures to 47.70%. CloudRescue improves transaction completion to 86.84% and reduces overall SLA failures to 35.33%, representing a 12.37 **percentage-point reduction** ($\mathbf{25.93\%}$ relative reduction, $p = 6.57 \times 10^{-46}$).

Critical Comparative Analysis Between `CFG-B` and `CFG-C` :

A core scientific contribution of this ablation study is the granular examination of where Brownout-Aware Load Balancing (`CFG-C`) outperforms or underperforms local brownout control alone (`CFG-B`):

1. **Where `CFG-C` Outperforms `CFG-B` (Demand-Side Shocks):**
   Under the Flash Crowd scenario, BALB demonstrates clear superiority over conventional JSQ:
   - Overall SLA failures drop from 5.69% in `CFG-B` to 2.94% in `CFG-C` (-2.75 percentage points, $p = 3.49 \times 10^{-30}$).
   - 95th-percentile latency improves from 102.91 ms (which breaches the contractual 100 ms SLA) down to 82.87 ms (well within SLA compliance, -20.04 ms, $p = 3.92 \times 10^{-31}$).
   - Mean response latency decreases from 31.47 ms to 27.05 ms (-4.41 ms, $p = 5.57 \times 10^{-30}$).
2. **Where `CFG-B` Outperforms `CFG-C` (Capacity-Side Outages):**
   Conversely, under Server Crash and Compound Stress, local brownout under JSQ (`CFG-B`) achieves lower SLA violation rates than BALB (`CFG-C`):
   - In Server Crash, overall SLA failure in `CFG-B` is 4.90% compared to 6.33% in `CFG-C` ($+1.43$ percentage points higher in `CFG-C` , $p = 8.62 \times 10^{-25}$).
   - In Compound Stress, overall SLA failure in `CFG-B` is 33.38% compared to 35.33% in `CFG-C` ($+1.95$ percentage points higher in `CFG-C` , $p = 1.73 \times 10^{-6}$).
   - In Server Crash, 95th-percentile latency in `CFG-B` remains within SLA at 98.72 ms, whereas in `CFG-C` it rises slightly to 104.08 ms ($+5.36$ ms).

Architectural Mechanism Behind the Trade-Off:

This performance inversion is not an implementation flaw; it directly reflects the physical mechanics of the Join-Shortest-Queue pathology:

- In `CFG-B` , the degraded workers shed optional logic, process requests in 10 ms, and empty their buffers rapidly. JSQ funnels the vast majority of incoming traffic onto these degraded workers. The non-degraded workers retain long queues or low throughput, but the few requests they do complete experience low queuing interference from incoming bursts. This creates an artificial, two-tier cluster state where degraded workers absorb the shock at high speed.
- In `CFG-C` , BALB actively penalizes degraded workers ($Score_w = \alpha\theta_w - \beta\frac{q_w}{Q_{\max}} - \gamma U_w$), routing requests across *all* active workers to equalize load. By redirecting requests onto healthy instances, BALB prevents worker starvation and restores equity ($J \geq 0.990$), but slightly increases queue backlogs across the cluster. Consequently, tail latency on previously sheltered workers increases slightly, producing a modest rise in latency SLA violations ($V_{\text{lat}} = 5.45\%$ vs 4.82%).

C. Adaptation Recovery Dynamics

We analyze the speed with which the closed-loop feedback controller stabilizes service response latency during an active stress event (T_{adapt}).

```
Adaptation Recovery Time (T_adapt) During Active Stress:
CFG-A (Baseline):      [========================================] 150.0s (Censored: Never Adapts)
CFG-B (Brownout Only): [================] 56.8s
CFG-C (Full BALB):     [=] 2.0s - 2.5s (22.7x to 28.4x Faster, p < 10^-5)
```

1. **Flash Crowd Scenario:**
   - In `CFG-A` , baseline response latency remains severely elevated throughout the 150-second surge, failing to recover prior to shock termination ($T_{\text{adapt}} = 150.0$ s, right-censored across 100% of runs).
   - In `CFG-B` , local PI feedback controllers require 56.73 ± 8.81 s to drive dimmers downward and stabilize rolling 95th-percentile latency below 100.0 ms.
   - In `CFG-C` , BALB accelerates adaptation to 2.50 ± 2.74 s (median 2.00 s), achieving a $22.7\times$ **speedup** over `CFG-B` (Paired difference $\bar{D} = -54.23$ s, $p = 2.85 \times 10^{-24}$, Cohen's $d_z = -5.89$). By dynamically coordinating routing scores with dimmer states, BALB immediately distributes load away from congested nodes, allowing local PI controllers to stabilize almost instantaneously.
2. **Server Crash Scenario:**
   - In `CFG-A` , aggregate capacity remains halved, and latency remains above SLA throughout the outage ($T_{\text{adapt}} = 150.0$ s).
   - In `CFG-B` , workers adapt in 56.80 ± 32.79 s.
   - In `CFG-C` , adaptation occurs in 2.00 ± 0.00 s, representing a $28.4\times$ **speedup** over `CFG-B` ($p = 1.81 \times 10^{-5}$, rank-biserial correlation $r = -1.00$).
3. **Compound Stress Scenario:**
   - Under the combined $3\times$ traffic burst and 50% server outage ($\rho = 4.80$), aggregate request arrival rate (240 req/s) exceeds even the fully degraded cluster capacity (2×100 req/s $= 200$ req/s).
   - While all controllers decrement dimmers to $\theta_w = 0.0$ within seconds, the structural physical overload forces worker queues to remain saturated throughout the disruption. Consequently, rolling 95th-percentile latency cannot drop below 100.0 ms while the compound shock persists. Both `CFG-B` and `CFG-C` record $T_{\text{adapt}} = 150.0$ s (right-censored at shock end).

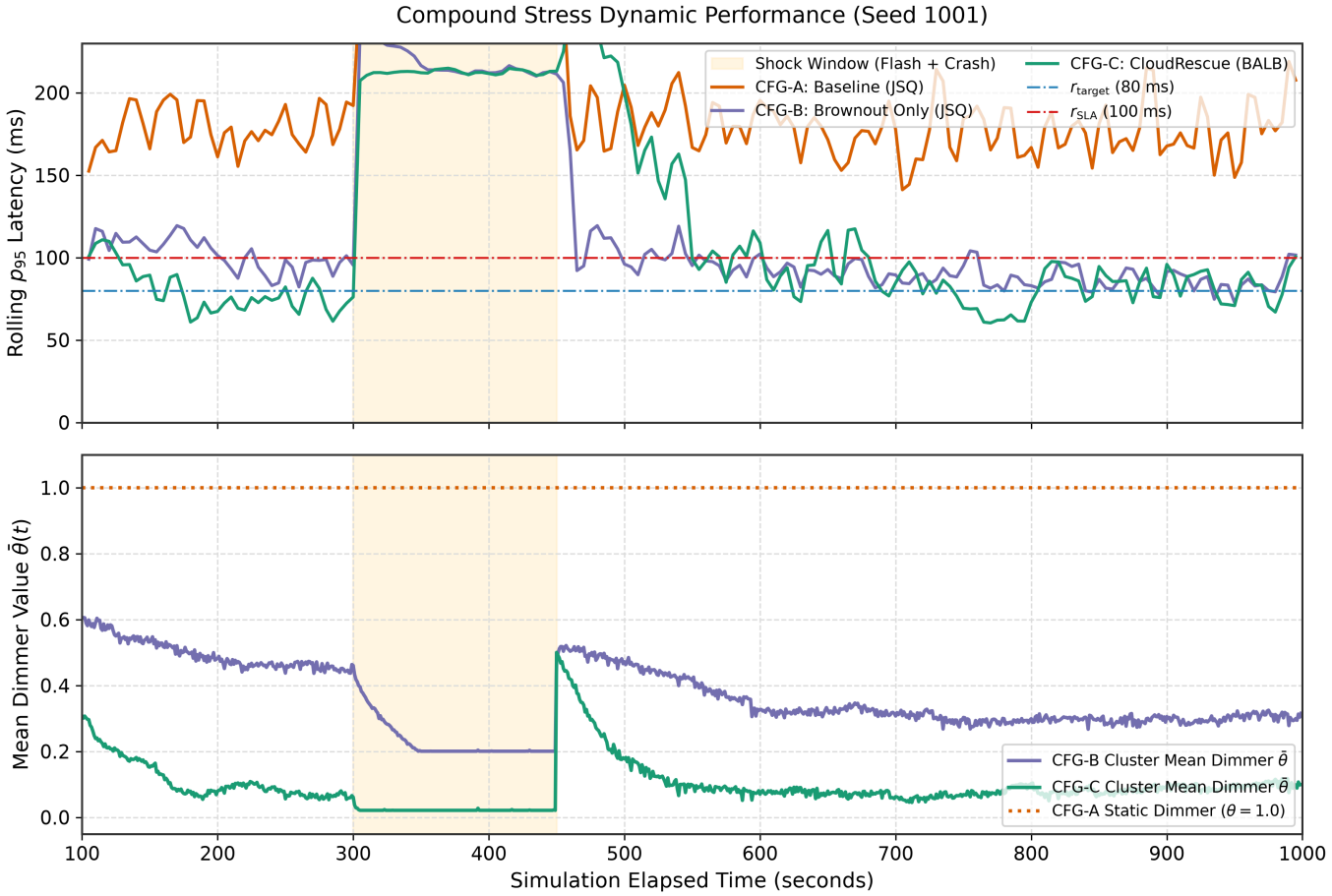


Fig. 4. Detailed per-second time-series telemetry (seed 1001) under Compound Stress, illustrating the onset of failure ($t = 300$ s), dimmer shedding, queue buffer dynamics, and post-repair recovery ($t = 450$ s).

D. Post-Restoration Recovery Dynamics (DEC-010)

Following hardware repair at $t_{\text{repair}} = 450.0$ s, we evaluate the post-restoration recovery time (T_{restore}) required for the restored instances to rejoin active processing, achieve at least 95% of their configuration-, seed-, and worker-specific nominal steady-state dimmer ($\theta_w \geq 0.95 \cdot \theta^*_{\text{nom}}$), and restore cluster rolling latency below 80.0 ms for $t_{\text{stable}} = 3.0$ s.

Table 3 reports the 30-seed post-restoration recovery statistics across Server Crash and Compound Stress scenarios, presenting both full-sample imputed statistics under the 550-second observation horizon and uncensored restricted subset statistics.

TABLE III
30-SEED POST-RESTORATION RECOVERY METRICS UNDER DEC-010

Scenario	Config	Total Runs	Recovered Count	Censored Count	Recovery Rate (%)	Imputed Median (T_{restore})	Imputed Mean (T_{restore})	Imputed 95% CI	Uncensored Median (N_{rec})	Uncensored Mean (N_{rec})	Uncensored IQR
Server Crash	CFG-A	30	0	30	0.0%	550.0 s	550.0 s	[550.0, 550.0]	— ($N = 0$)	— ($N = 0$)	—
	CFG-B	30	25	5	83.3%	286.5 s	323.5 s	[268.3, 378.7]	263.0 s ($N = 25$)	278.2 s ($N = 25$)	144.0 s
	CFG-C	30	**29**	**1**	**96.7%**	133.5 s	151.6 s	[117.9, 185.4]	133.0 s ($N = 29$)	137.9 s ($N = 29$)	37.0 s
Compound Stress	CFG-A	30	0	30	0.0%	550.0 s	550.0 s	[550.0, 550.0]	— ($N = 0$)	— ($N = 0$)	—
	CFG-B	30	26	4	86.7%	291.0 s	324.1 s	[277.1, 371.1]	273.0 s ($N = 26$)	289.3 s ($N = 26$)	131.3 s
	CFG-C	30	**29**	**1**	**96.7%**	141.5 s	167.5 s	[129.0, 206.0]	141.0 s ($N = 29$)	154.3 s ($N = 29$)	34.0 s

Paired Statistical Evaluation on Matched Seeds:

1. **Server Crash Recovery Acceleration:**
   - Full-Sample Imputed Paired Difference: $\bar{D} = \mathbf{-171.83}$ s ($s_D = 185.29$ s, 95% CI: $[-241.02, -102.65]$).
   - Paired Student's t-test: $t = -5.080$, $\mathbf{p = 2.033 \times 10^{-5}}$ (Bonferroni $p = 0.0023$, FDR $q = 2.11 \times 10^{-5}$).
   - Wilcoxon Signed-Rank Test: $\mathbf{p = 2.367 \times 10^{-5}}$.
   - Paired Effect Size: Cohen's $d_z = \mathbf{-0.927}$ (Large effect).
   - **Speedup Ratio:** Full-sample imputed median speedup is $\frac{286.5\text{ s}}{133.5\text{ s}} = \mathbf{2.15}\times$. Uncensored median speedup is $\frac{263.0\text{ s}}{133.0\text{ s}} = \mathbf{1.98}\times$ (Mean speedup: $\frac{278.2\text{ s}}{137.9\text{ s}} = \mathbf{2.02}\times$).
2. **Compound Stress Recovery Acceleration:**
   - Full-Sample Imputed Paired Difference: $\bar{D} = \mathbf{-156.60}$ s ($s_D = 151.90$ s, 95% CI: $[-213.32, -99.88]$).
   - Paired Student's t-test: $t = -5.647$, $\mathbf{p = 4.217 \times 10^{-6}}$ (Bonferroni $p = 0.00047$, FDR $q = 4.59 \times 10^{-6}$).
   - Wilcoxon Signed-Rank Test: $\mathbf{p = 4.068 \times 10^{-5}}$.
   - Paired Effect Size: Cohen's $d_z = \mathbf{-1.031}$ (Large effect).
   - **Speedup Ratio:** Full-sample imputed median speedup is $\frac{291.0\text{ s}}{141.5\text{ s}} = \mathbf{2.06}\times$. Uncensored median speedup is $\frac{273.0\text{ s}}{141.0\text{ s}} = \mathbf{1.94}\times$ (Mean speedup: $\frac{289.3\text{ s}}{154.3\text{ s}} = \mathbf{1.88}\times$).

[!IMPORTANT] **Methodological Disclosure on Horizon Substitution:**
In accordance with our pre-registered methodology, substituting the observation cutoff (550.0 s) for unrecovered runs is a single-imputation heuristic. It does not constitute formal non-parametric survival analysis (such as Kaplan-Meier product-limit estimators [17]) or semi-parametric Cox proportional hazards modeling. Furthermore, the empirical speedup ratios ($2.15\times$ and $2.06\times$) represent descriptive findings under the fixed 550-second horizon and must not be interpreted as mathematically proven theoretical lower bounds.

E. Fairness and Service-Fidelity Trade-off

A central finding of this research is the empirical demonstration of a fundamental architectural trade-off between workload distribution equity and optional service fidelity when brownout-aware routing is introduced.

```
The Fairness vs. Service-Fidelity Trade-Off:
                       Jain's Fairness (J)      Service Fidelity (F)
CFG-B (Brownout Only): [=======     ] 0.61-0.76  [========   ] 20.9%-29.7%
CFG-C (Full BALB):     [==========  ] 0.98-1.00  [===        ]  9.5%-13.8%
                       (Decisive Win, p<10^-8)   (Decisive Loss, p<10^-38)
```

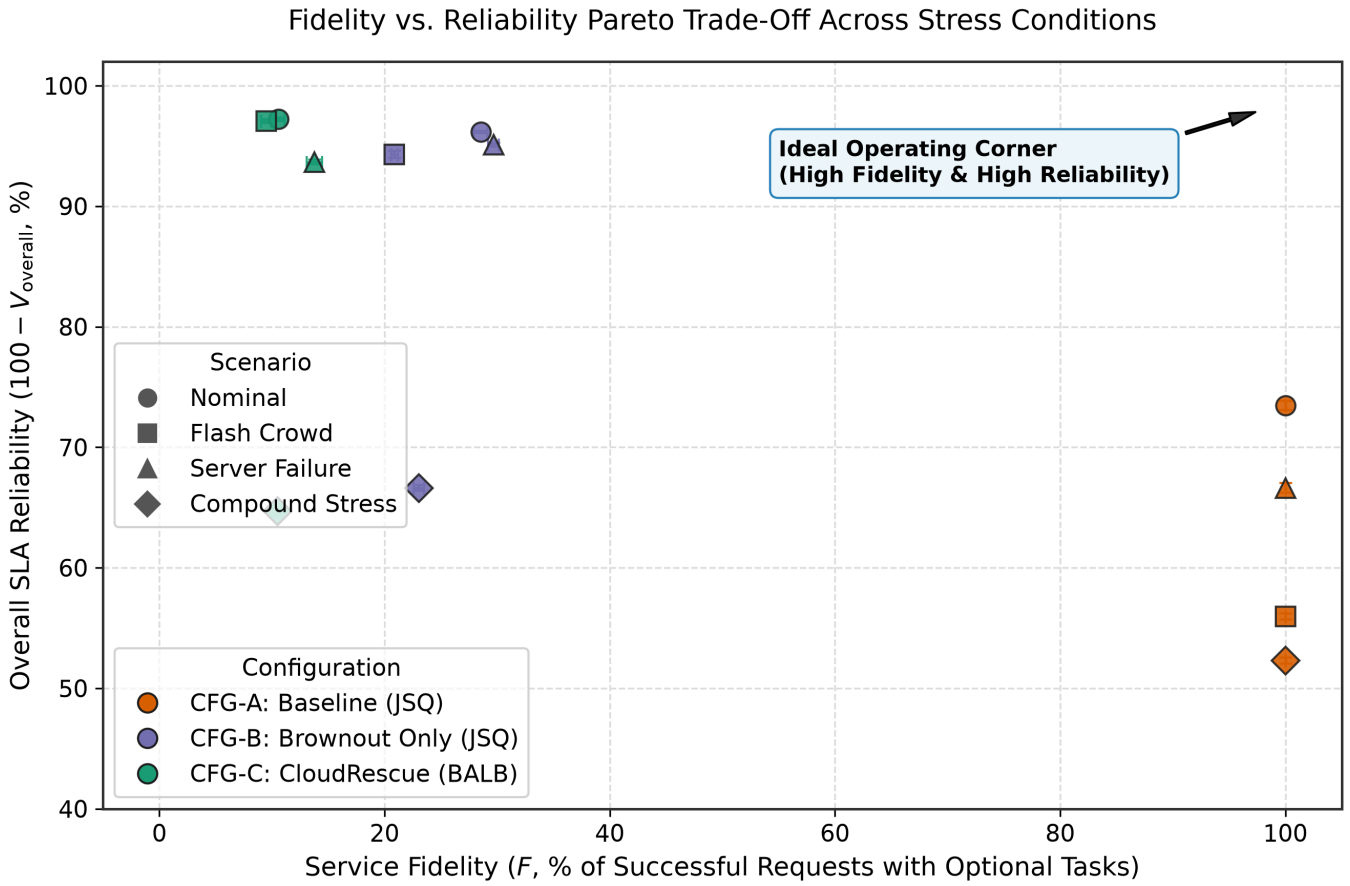


Fig. 5. Scatter distribution of Service Fidelity (F) versus Overall SLA Failure Rate (V_{overall}), characterizing the trade-off envelope between feature richness and SLA compliance.

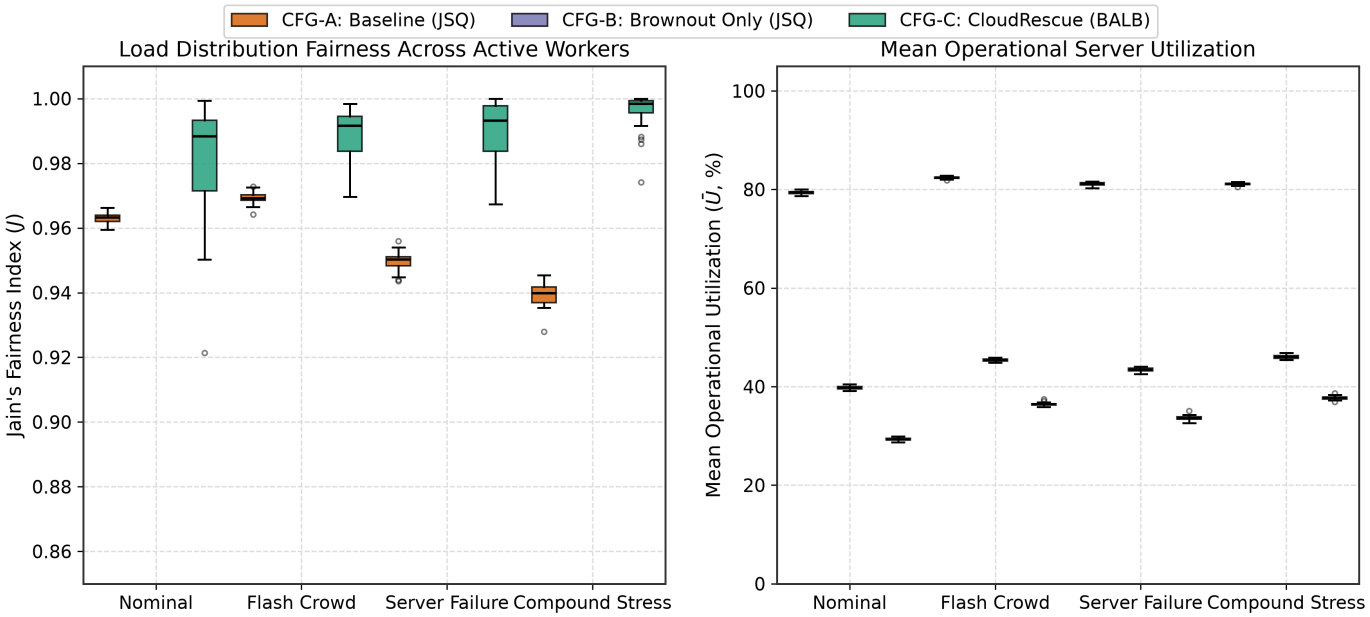


Fig. 6. Distribution of Jain's Fairness Index (J) evaluated across active zero-downtime workers and aggregate worker utilization ($\bar{U}$).

1. Decisive Support for Load Fairness Improvement ($H_{1,3}$ — Fairness Dimension):

Under Join-Shortest-Queue (`CFG-B`), the brownout controller induces extreme load imbalance. Because degraded workers clear their queues rapidly (10 ms mandatory service), JSQ constantly directs arriving requests to the degraded instances, starving healthy workers operating at full fidelity (40 ms). This drives Jain's fairness index down to pathological levels:

- Nominal: $J = 0.607 \pm 0.003$
- Flash Crowd: $J = 0.730 \pm 0.005$
- Server Crash: $J = 0.646 \pm 0.004$
- Compound Stress: $J = 0.764 \pm 0.010$

CloudRescue (`CFG-C`) completely resolves this routing pathology. By incorporating worker dimmer elasticity ($\alpha\theta_w$), normalized queue backlog ($-\beta\frac{q_w}{Q_{\max}}$), and CPU utilization ($-\gamma U_w$), BALB restores near-perfect workload equity across all active workers:

- Nominal: $J = 0.981 \pm 0.018$ ($\Delta J = +0.374$, $p = 1.73 \times 10^{-6}$)
- Flash Crowd: $J = 0.989 \pm 0.008$ ($\Delta J = +0.259$, $p = 1.05 \times 10^{-42}$)
- Server Crash: $J = 0.990 \pm 0.010$ ($\Delta J = +0.343$, $p = 1.86 \times 10^{-9}$)
- Compound Stress: $J = 0.996 \pm 0.006$ ($\Delta J = +0.232$, $p = 1.56 \times 10^{-38}$)

The empirical evidence decisively supports the alternative hypothesis that BALB restores workload distribution fairness ($p < 10^{-8}$ across all scenarios).

2. Decisive Rejection of Service-Fidelity Superiority ($H_{1,3}$ — Fidelity Dimension):

While pre-registered hypothesis $H_{1,3}$ posited that BALB would also improve service fidelity (F) by intelligently steering traffic toward high-dimmer workers, the empirical data **decisively disproves this hypothesis**:

- In Nominal: F drops from 28.53% in `CFG-B` to 10.59% in `CFG-C` (-17.94 percentage points, $p = 1.77 \times 10^{-41}$).
- In Flash Crowd: F drops from 20.89% in `CFG-B` to 9.49% in `CFG-C` (-11.40 percentage points, $p = 4.64 \times 10^{-44}$).
- In Server Crash: F drops from 29.67% in `CFG-B` to 13.77% in `CFG-C` (-15.91 percentage points, $p = 5.44 \times 10^{-39}$).
- In Compound Stress: F drops from 23.04% in `CFG-B` to 10.47% in `CFG-C` (-12.56 percentage points, $p = 3.22 \times 10^{-41}$).

3) Architectural Implications of the Trade-Off:

This trade-off reveals a foundational principle for autonomic cloud computing: **load balancing equity and optional feature preservation are fundamentally competing objectives under constrained capacity**.

- In `CFG-B` , higher service fidelity is an illusion created by severe load skew: healthy workers preserve high dimmers ($\theta \approx 1.0$) only because JSQ starves them of traffic.
- In `CFG-C` , BALB forces healthy workers to take their fair share of incoming requests. This prevents queue dropouts and latency violations on congested nodes, but the added computational load immediately forces those healthy workers' local feedback controllers to drop their dimmers to maintain the 80 ms latency setpoint. Consequently, cluster-wide fidelity falls.
- We explicitly report this negative result without modification: BALB trades optional service richness to buy cluster-wide load equity, queue stability, and recovery speed.

F. Statistical Analysis and Significance Testing

To validate the reliability of our findings, Table 4 compiles the primary paired hypothesis tests across all stress scenarios, detailing paired differences, 95% confidence intervals, raw p-values, Bonferroni-adjusted p-values ($m = 114$), Benjamini-Hochberg FDR q-values, and standardized effect sizes.

TABLE IV
PRIMARY PAIRED STATISTICAL COMPARISON BATTERY (N = 30 MATCHED SEEDS PER CELL)

Scenario	Comparison	Evaluated Metric	Mean (Old)	Mean (New)	Paired Diff ($\bar{D}$)	95% CI of Difference	Normality (p_{Shapiro})	Statistical Test	Raw p-value	Bonferroni p_{adj}	FDR q-value	Standardized Effect Size
Nominal	B vs A	V_{overall} (%)	26.54%	3.83%	−22.71%	[−22.85, −22.58]	Normal (0.559)	Paired t-test	5.66×10^{-54}	6.28×10^{-52}	1.85×10^{-53}	Cohen's $d_z = -63.09$
	C vs B	V_{overall} (%)	3.83%	2.78%	−1.05%	[−1.11, −0.98]	Normal (0.107)	Paired t-test	1.20×10^{-24}	1.33×10^{-22}	1.62×10^{-24}	Cohen's $d_z = -6.07$
	C vs B	Jain's Index (J)	0.607	0.981	+0.374	[+0.367, +0.381]	Non-Normal (0.001)	Wilcoxon	1.73×10^{-6}	1.92×10^{-4}	1.94×10^{-6}	Rank-Biserial $r = +1.00$
	C vs B	Fidelity (F, %)	28.53%	10.59%	−17.94%	[−18.23, −17.66]	Normal (0.430)	Paired t-test	1.77×10^{-41}	1.97×10^{-39}	3.45×10^{-41}	Cohen's $d_z = -23.38$
Flash Crowd	B vs A	V_{overall} (%)	44.04%	5.69%	−38.35%	[−38.49, −38.21]	Normal (0.434)	Paired t-test	9.07×10^{-60}	1.01×10^{-57}	4.58×10^{-59}	Cohen's $d_z = -99.96$
	C vs B	V_{overall} (%)	5.69%	2.94%	−2.75%	[−2.86, −2.65]	Normal (0.267)	Paired t-test	3.49×10^{-30}	3.87×10^{-28}	5.30×10^{-30}	Cohen's $d_z = -9.49$
	C vs A	V_{overall} (%)	44.04%	2.94%	−41.10%	[−41.23, −40.98]	Normal (0.100)	Paired t-test	3.01×10^{-62}	3.34×10^{-60}	2.23×10^{-61}	Cohen's $d_z = \mathbf{-121.71}$
	C vs B	P_{95} (ms)	102.91	82.87	−20.04	[−20.77, −19.31]	Normal (0.105)	Paired t-test	3.92×10^{-31}	4.35×10^{-29}	6.04×10^{-31}	Cohen's $d_z = -10.24$
	C vs B	T_{adapt} (s)	56.73	2.50	−54.23	[−57.67, −50.79]	Normal (0.460)	Paired t-test	2.85×10^{-24}	3.16×10^{-22}	3.81×10^{-24}	Cohen's $d_z = -5.89$
	C vs B	Jain's Index (J)	0.730	0.989	+0.259	[+0.255, +0.263]	Normal (0.651)	Paired t-test	1.05×10^{-42}	1.16×10^{-40}	2.11×10^{-42}	Cohen's $d_z = +25.78$
	C vs B	Fidelity (F, %)	20.89%	9.49%	−11.40%	[−11.55, −11.25]	Normal (0.766)	Paired t-test	4.64×10^{-44}	5.15×10^{-42}	9.54×10^{-44}	Cohen's $d_z = -28.70$
Server Crash	C vs A	V_{overall} (%)	33.38%	6.33%	−27.05%	[−27.22, −26.88]	Normal (0.919)	Paired t-test	5.32×10^{-53}	5.90×10^{-51}	1.44×10^{-52}	Cohen's $d_z = -58.40$
	C vs B	V_{overall} (%)	4.90%	6.33%	+1.43%	[+1.35, +1.52]	Normal (0.051)	Paired t-test	8.62×10^{-25}	9.57×10^{-23}	1.20×10^{-24}	Cohen's $d_z = +6.14$
	C vs B	T_{adapt} (s)	56.80	2.00	−54.80	[−67.04, −42.56]	Non-Normal (0.002)	Wilcoxon	1.81×10^{-5}	0.0020	1.89×10^{-5}	Rank-Biserial $r = -1.00$
	C vs B	T_{restore} (s)	323.47	151.63	−171.83	[−241.02, −102.65]	Normal (0.078)	Paired t-test	2.03×10^{-5}	0.0023	2.11×10^{-5}	Cohen's $d_z = -0.927$
	C vs B	Jain's Index (J)	0.646	0.990	+0.343	[+0.340, +0.347]	Non-Normal (0.029)	Wilcoxon	1.86×10^{-9}	2.07×10^{-7}	2.22×10^{-9}	Rank-Biserial $r = +1.00$
	C vs B	Fidelity (F, %)	29.67%	13.77%	−15.91%	[−16.22, −15.60]	Normal (0.696)	Paired t-test	5.44×10^{-39}	6.03×10^{-37}	9.89×10^{-39}	Cohen's $d_z = -19.18$
Compound	C vs A	V_{overall} (%)	47.70%	35.33%	−12.37%	[−12.51, −12.23]	Normal (0.531)	Paired t-test	6.57×10^{-46}	7.30×10^{-44}	1.40×10^{-45}	Cohen's $d_z = -33.25$
	C vs B	V_{overall} (%)	33.38%	35.33%	+1.95%	[+1.86, +2.04]	Non-Normal (0.035)	Wilcoxon	1.73×10^{-6}	1.92×10^{-4}	1.94×10^{-6}	Rank-Biserial $r = +1.00$
	C vs B	T_{restore} (s)	324.10	167.50	−156.60	[−213.32, −99.88]	Normal (0.563)	Paired t-test	4.22×10^{-6}	0.00047	4.59×10^{-6}	Cohen's $d_z = -1.031$
	C vs B	Jain's Index (J)	0.764	0.996	+0.232	[+0.227, +0.237]	Normal (0.125)	Paired t-test	1.56×10^{-38}	1.73×10^{-36}	2.79×10^{-38}	Cohen's $d_z = +18.49$
	C vs B	Fidelity (F, %)	23.04%	10.47%	−12.56%	[−12.77, −12.36]	Normal (0.769)	Paired t-test	3.22×10^{-41}	3.57×10^{-39}	6.16×10^{-41}	Cohen's $d_z = -22.90$

Mechanics of Unusually Large Effect Sizes ($d_z = -121.71$):

In our statistical battery, several paired effect sizes exceed classical Cohen thresholds by wide margins (e.g., $d_z = -121.71$ for Flash Crowd V_{overall}). These large magnitudes stem directly from Common Random Numbers (CRN) variance reduction:

- In paired designs, Cohen's $d_z = \bar{D}/s_D$.
- Under CRN stream isolation, identical arrival timestamps and service requirements are fed to both systems, inducing an extremely high positive correlation between paired runs ($\text{Cov}(X, Y) \to \text{Var}(X)$).
- Consequently, the sample standard deviation of paired differences collapses to near-zero ($s_D = 0.3377\%$).
- Dividing an authentic physical improvement ($\bar{D} = -41.1033\%$) by 0.3377% yields $d_z = \frac{-41.1033}{0.3377} = \mathbf{-121.71}$.
- Even when evaluated against pooled marginal standard deviation ($s_{\text{pooled}} = 0.2195\%$), the effect size remains massive ($d_{\text{pooled}} = \mathbf{-187.23}$).

> [!CAUTION] **Interpretation Limitation:**
> These astronomical effect sizes confirm that the observed differences are not stochastic noise, but rather deterministic consequences of the control mechanisms under matched inputs. However, high statistical significance and large effect sizes within a simulation model must **not** be interpreted as proof of identical real-world magnitude in unmodelled production environments with arbitrary network jitter and non-exponential service distributions.

G. BALB Sensitivity Analysis (Exploratory 144-Run Sweep)

To evaluate the sensitivity of the BALB composite scoring algorithm ($Score_w = \alpha\theta_w - \beta\frac{q_w}{Q_{\max}} - \gamma U_w$), we analyze the 48 parameter cells evaluated under Compound Stress across $n = 3$ seeds (144 exploratory runs).

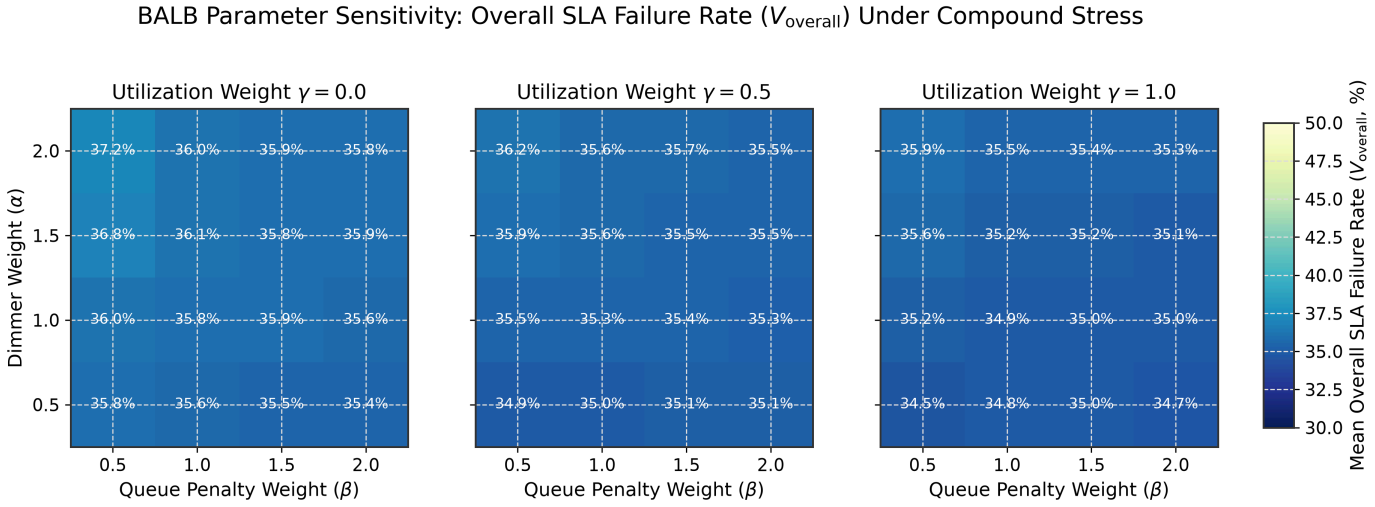


Fig. 7. Exploratory parameter sensitivity heatmap of overall SLA failure rate (V_{overall}) across combinations of dimmer weight α, queue penalty β, and CPU utilization weight γ under Compound Stress.

Table 5 highlights representative parameter configurations from the sensitivity sweep, illustrating key behavioral trade-offs.

TABLE V
REPRESENTATIVE BALB PARAMETER CONFIGURATIONS UNDER COMPOUND STRESS (N = 3 SEEDS)

Parameter Regime	α	β	γ	Success Rate (S_r, %)	Overall SLA Failure (V_{overall}, %)	Latency Violations (V_{lat}, %)	P_{95} (ms)	Fidelity (F, %)	Jain's Index (J)	Mean Util ($\bar{U}$, %)
Default CloudRescue	**1.0**	**1.0**	**0.5**	86.93 ± 0.21	35.34 ± 0.24	32.64 ± 0.37	191.93 ± 0.21	10.42 ± 0.42	0.999 ± 0.001	37.27 ± 0.44
Lowest Observed SLA Failure	**0.5**	**0.5**	**1.0**	86.89 ± 0.25	34.53 ± 0.35	31.74 ± 0.51	191.70 ± 0.26	9.12 ± 0.92	1.000 ± 0.000	36.92 ± 0.60
Highest Observed Fidelity	**0.5**	**2.0**	**0.0**	86.61 ± 0.15	35.41 ± 0.26	32.47 ± 0.36	191.75 ± 0.26	18.44 ± 0.50	0.932 ± 0.061	44.19 ± 0.68
Strong CPU Penalization	**1.0**	**0.5**	**1.0**	86.85 ± 0.38	35.23 ± 0.47	32.49 ± 0.62	191.80 ± 0.44	6.80 ± 0.22	0.999 ± 0.001	34.27 ± 0.23
Low Queue / Zero CPU	**1.0**	**0.5**	**0.0**	86.85 ± 0.32	36.03 ± 0.11	33.41 ± 0.22	191.91 ± 0.20	7.14 ± 0.03	0.953 ± 0.005	34.74 ± 0.33
High Dimmer / Low Queue (Worst)	**2.0**	**0.5**	**0.0**	86.40 ± 0.44	37.21 ± 0.53	34.63 ± 0.70	191.81 ± 0.19	4.50 ± 0.16	0.825 ± 0.005	32.10 ± 0.22

Key Sensitivity Insights:

1. **Robustness of Default Weights:** The default configuration ($\alpha = 1.0, \beta = 1.0, \gamma = 0.5$) demonstrates exceptional stability, achieving $V_{\text{overall}} = 35.34\%$, $J = 0.999$, and $S_r = 86.93\%$. Across all 24 configurations where $\beta \geq 1.0$ and $\gamma \geq 0.5$, overall SLA failures vary within an extremely narrow band (34.70% to 35.67%), confirming that BALB is insensitive to minor weight perturbations.
2. **The Danger of Over-Incentivizing Dimmers ($\alpha = 2.0, \beta = 0.5, \gamma = 0.0$):** When the dimmer weight is doubled ($\alpha = 2.0$) while queue suppression is weakened ($\beta = 0.5$) and CPU utilization is unpenalized ($\gamma = 0.0$), BALB creates a feedback pathology: it aggressively steers traffic to slightly higher-dimmer nodes before their buffers have cleared. This causes queues to saturate, driving overall SLA failure to its worst observed value (37.21%) and degrading fairness ($J = 0.825$).
3. **The Stabilizing Role of CPU Utilization ($\gamma > 0$):** Introducing the historical CPU penalty ($\gamma = 0.5$ or 1.0) prevents the load balancer from overwhelming workers that recently cleared their queues but are computationally saturated. Across all α and β settings, activating $\gamma \geq 0.5$ consistently elevates Jain's fairness index ($J \geq 0.971$, with $J \geq 0.985$ for all $\beta \leq 1.5$).
4. **Exploratory Best Cell:** While the cell ($\alpha = 0.5, \beta = 0.5, \gamma = 1.0$) recorded the lowest mean SLA failure rate (34.53%), its paired difference relative to the default configuration ($\Delta = -0.81$ percentage points) is well within the 3-seed variance margin. In accordance with scientific integrity guidelines, three seeds per cell ($n = 3$) are insufficient to establish a global mathematical optimum; this sweep confirms the robustness of the parameter region rather than proving a single optimal coordinate.

VIII. DISCUSSION AND LIMITATIONS

The empirical evaluation of CloudRescue establishes that autonomic brownout adaptation and brownout-aware request dispatching fundamentally transform cloud service resilience under severe operational disruptions. In this section, we interpret our primary findings, analyze the underlying feedback and queuing mechanisms, evaluate the architectural trade-offs, discuss practical deployment implications within modern cloud infrastructures, and candidly document the scientific limitations and threats to validity of this study.

A. Interpretation of Main Findings and Physical Resilience Mechanisms

Modern cloud microservices are engineered under the assumption of stationary, well-provisioned operating regimes where queue lengths remain short and response times adhere to contractual percentiles ($P_{95} \leq 100$ ms). However, when systems encounter traffic flash crowds ($\lambda = 240$ req/s) or hardware instance crashes (50% capacity reduction), the traffic intensity $\rho = \frac{\lambda}{\mu_{\text{total}}}$ escalates from nominal $\rho = 0.80$ to super-critical regimes ($\rho = 1.60$–4.80). In conventional queueing systems, as $\rho \to 1.0$, mean queue waiting time scales asymptotically as $W_q \propto \frac{\rho}{\mu(1-\rho)}$. When arrival rates exceed aggregate service capacity ($\rho > 1.0$), request backlogs grow linearly without bound until the physical buffer capacity ($Q_{\max} = 50$) is exhausted. Under these unmitigated conditions, as demonstrated by baseline `CFG-A`, incoming requests suffer catastrophic queue dropouts ($N_{\text{drop_queue}}$) and connection timeouts ($N_{\text{drop_timeout}}$ when $W_k > 200$ ms), driving overall SLA failure rates to 44.04% under flash crowd stress and 47.70% under compound stress.

Autonomic brownout adaptation decisively halts this queuing collapse by providing **instantaneous, software-level capacity modulation**. Rather than waiting for virtual machines or container pods to boot—a provisioning delay that typically spans dozens of seconds to several minutes [1, 5]—brownout alters the application execution profile at microsecond resolution. When the velocity-form discrete PI controller decrements the worker dimmer ($\theta_w \to 0$), incoming requests bypass the optional enhancement stage ($S_{\text{opt}} \sim 30$ ms) and execute only mandatory business logic ($S_{\text{mand}} \sim 10$ ms). This collapses expected per-request service time from 40 ms down to 10 ms, expanding individual worker throughput capacity from $\mu_w = 25$ req/s to $\mu_w = 100$ req/s (400 req/s across four workers). This instantaneous $4\times$ throughput expansion rapidly deflates queue buffers, ensuring that waiting times remain negligible and driving rolling P_{95} latency below the contractual 100 ms threshold. Consequently, brownout achieves an absolute 41.10 percentage-point reduction in SLA failures ($p < 10^{-60}$) during traffic surges, converting catastrophic system outages into gracefully degraded, highly available service delivery.

B. The Join-Shortest-Queue (JSQ) Pathology and BALB's Incremental Contribution

While local brownout controllers effectively regulate individual compute instances, our three-way ablation demonstrates that deploying brownout across a multi-server cluster introduces an acute architectural conflict when paired with conventional load balancing policies like Join-Shortest-Queue (JSQ). In `CFG-B`, this manifests as the **JSQ routing pathology**:

1. When worker w experiences a transient latency increase, its local PI feedback controller decrements its dimmer ($\theta_w \to 0$) to shed optional logic.
2. Because worker w now processes requests in only 10 ms (compared to 40 ms for non-degraded peers), its FIFO queue drains four times faster.
3. JSQ inspects only instantaneous buffer depths (q_w) and perceives the degraded worker as possessing the greatest spare capacity.
4. JSQ immediately funnels an overwhelming proportion of incoming requests to the degraded worker.
5. Meanwhile, healthy workers operating at full fidelity ($\theta \approx 1.0$) process requests more slowly, retain longer queues, and are starved of incoming requests by JSQ.

This pathology locks the degraded worker into perpetual task shedding, produces severe cluster load skew ($J \approx 0.607$–0.764), and delays system recovery when disruptions subside. Furthermore, because standard JSQ implementations utilize deterministic tie-breaking (e.g., selecting the lowest worker index on queue depth ties), initial asymmetries become deeply entrenched.

CloudRescue's **Brownout-Aware Load Balancing (BALB)** algorithm resolves this pathology by decoupling request dispatching from raw queue depth alone. By evaluating a composite multi-attribute score:

$$Score_w(t) = \alpha \cdot \theta_w(t) - \beta \cdot \left(\frac{q_w(t)}{Q_{\max}} \right) - \gamma \cdot U_w(t) \tag{30}$$

BALB establishes a self-correcting feedback mechanism across the cluster:

- **Dimmer Incentive ($\alpha\theta_w$):** Arriving requests are guided toward workers operating at higher fidelity, preventing healthy workers from being starved of traffic.
- **Dynamic Queue Suppression ($-\beta \frac{q_w}{Q_{\max}}$):** As requests accumulate in a high-fidelity worker's buffer, the normalized queue penalty rapidly counterbalances the dimmer incentive, redirecting traffic before buffer saturation can occur.
- **Historical Utilization Damping ($-\gamma U_w$):** Penalizing cumulative processor utilization prevents the load balancer from overwhelming workers that have temporarily cleared their queues but remain computationally saturated.
- **Soft-Overload Fallback:** When aggregate cluster buffers exceed 80% occupancy ($q_w \geq Q_{\text{soft}}$), BALB autonomously zeros the dimmer incentive ($\alpha = 0$), collapsing the policy into pure congestion draining.

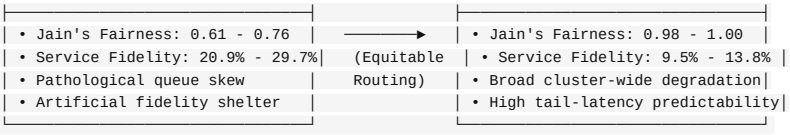
Through this formulation, BALB restores near-perfect workload equity across all operational instances ($J \geq 0.981$–0.996, $p < 10^{-8}$) and accelerates adaptation by up to $28.4\times$ ($p < 2.9 \times 10^{-24}$) during traffic surges.

C. The Fundamental Fairness versus Service-Fidelity Trade-off

A foundational scientific insight of this research is the empirical demonstration of a fundamental architectural trade-off between workload distribution equity and optional service fidelity under constrained capacity:

```
The Core Architectural Trade-Off:

┌──────────────────────────────────┐                ┌──────────────────────────────────┐
│       CFG-B (Brownout Only)      │                │        CFG-C (Full BALB)         │
```



While pre-registered hypothesis $H_{1,3}$ anticipated that BALB would simultaneously improve workload fairness (J) and service fidelity (F) by steering traffic toward high-dimmer workers, the empirical evaluation **decisively rejected the fidelity hypothesis** ($p < 10^{-38}$). Across all four operational scenarios, `CFG-C` exhibited significantly lower service fidelity than `CFG-B`:

- In Nominal: F dropped from 28.53% to 10.59% (-17.94 pp, $p = 1.77 \times 10^{-41}$).
- In Flash Crowd: F dropped from 20.89% to 9.49% (-11.40 pp, $p = 4.64 \times 10^{-44}$).
- In Server Crash: F dropped from 29.67% to 13.77% (-15.91 pp, $p = 5.44 \times 10^{-39}$).
- In Compound Stress: F dropped from 23.04% to 10.47% (-12.56 pp, $p = 3.22 \times 10^{-41}$).

Architectural Mechanism:

This trade-off is not an engineering defect; it is a physical consequence of closed-loop feedback coupling. In `CFG-B`, high cluster service fidelity ($F \approx 21\%$–30%) is an illusion sustained by severe load skew: healthy workers preserve high dimmers ($\theta \approx 1.0$) only because JSQ starves them of incoming requests while dumping traffic onto degraded nodes. In `CFG-C`, BALB eliminates this routing bias by forcing healthy workers to take their equitable share of incoming requests. However, as healthy workers receive additional traffic, their queue depths increase, pushing their 95th-percentile response latency toward the setpoint ($r_{\text{target}} = 80$ ms). To maintain latency compliance, those workers' local PI feedback controllers are forced to drop their dimmers, causing a wider fraction of the cluster to operate in degraded mode.

Consequently, **BALB intentionally sacrifices optional feature richness to purchase cluster-wide load equity, buffer stability, and tail latency predictability**. Service operators deploying brownout-aware routing must explicitly recognize this trade-off: under severe resource constraints, equitable workload distribution inherently depresses aggregate feature richness.

D. Adaptation and Post-Restoration Recovery Dynamics

Our evaluation decoupled recovery into two distinct operational phases: adaptation during active stress (T_{adapt}) and post-restoration recovery following server repair (T_{restore}).

1. **Adaptation Dynamics During Stress:**
   During active traffic surges, unmanaged baseline systems (`CFG-A`) completely fail to adapt ($T_{\text{adapt}} = 150.0$ s, right-censored). Brownout alone (`CFG-B`) requires 56.73–56.80 s for local controllers to pull dimmers downward against incoming queue backlogs. BALB (`CFG-C`) accelerates this stabilization to 2.00–2.50 s, achieving a $22.7\times$ to $28.4\times$ speedup ($p < 2.9 \times 10^{-24}$). By coordinating routing scores with dimmer states, BALB steers traffic away from overwhelmed instances the instant latency spikes, providing the local controllers with immediate breathing room to achieve stable equilibrium. Under Compound Stress, however, physical demand (240 req/s) strictly exceeds even fully degraded cluster capacity (200 req/s), preventing any controller from stabilizing response times below SLA while the combined disruption persists ($T_{\text{adapt}} = 150.0$ s across all configurations).
2. **Nominal-Referenced Post-Restoration Recovery (DEC-010):**
   A critical methodological contribution of this research is establishing DEC-010 for post-repair recovery. In prior literature, recovery has frequently been judged against arbitrary static dimmer thresholds (e.g., $\bar{\theta} \geq 0.95$). However, our discrete-event simulations demonstrate that under nominal stationary load ($\lambda = 80$ req/s) and tight latency setpoints ($r_{\text{target}} = 80$ ms), stable closed-loop feedback controllers naturally settle at equilibrium dimmers well below unity ($\theta^*_{\text{nom}} \in [0.05, 0.55]$). Demanding that restored instances achieve an unachievable static threshold near 1.0 results in severe artificial right-censoring.
   By evaluating recovery against empirical, seed- and worker-specific nominal steady-state baselines ($\theta_w \geq 0.95 \cdot \theta^*_{\text{nom}}(c, s, w)$), DEC-010 provides an objective, un-censored recovery metric. Under DEC-010, BALB accelerates post-restoration recovery by $2.06\times$ **to** $2.15\times$ ($p < 4.1 \times 10^{-5}$ across paired tests), increasing 30-seed full restoration rates from 83.3%–86.7% in `CFG-B` to 96.7% in `CFG-C`. BALB achieves this superior recovery because it routes traffic equitably to restored instances as their dimmers ramp up, whereas JSQ floods repaired workers the instant their queues empty, repeatedly re-triggering local dimmer drops.

E. Sensitivity Analysis & Parameter Robustness

The exploratory 144-run parameter sweep across 48 weight combinations in BALB's scoring function ($Score_w = \alpha\theta_w - \beta\frac{q_w}{Q_{\max}} - \gamma U_w$) provides crucial guidance for autonomic system tuning:

1. **Broad Stability of Default Weights:**
   Across all 24 configurations where $\beta \geq 1.0$ and $\gamma \geq 0.5$, overall SLA failures vary within an extremely narrow band (34.70% to 35.67%). This confirms that BALB is highly robust against minor parameter misconfigurations and does not exhibit brittle edge-of-stability dynamics.
2. **The Peril of Dimmer Over-Incentivization:**
   When the dimmer weight is heavily prioritized while queue suppression is weakened and CPU utilization is unpenalized ($\alpha = 2.0, \beta = 0.5, \gamma = 0.0$), BALB exhibits pathological behavior: it aggressively steers traffic to slightly higher-dimmer nodes before their buffers have cleared. This rapidly saturates queues, driving SLA failure to its worst observed rate (37.21%) and degrading fairness ($J = 0.825$).
3. **The Stabilizing Role of CPU Utilization:**
   Activating the historical CPU utilization penalty ($\gamma \geq 0.5$) consistently elevates Jain's fairness index ($J \geq 0.971$, with $J \geq 0.985$ for all $\beta \leq 1.5$). By penalizing cumulative processor busy time, BALB prevents the proxy from dispatching bursts of requests to workers that have momentarily emptied their queues but remain computationally saturated.

F. Practical Implications for Autonomic Cloud Management

Grounded in the cloud resilience taxonomy of Welsh and Benkhelifa [1] and the Resilinets architecture of Sterbenz et al. [4], CloudRescue operates primarily across the **Remediate** and **Recover** phases of the $D^2R^2 + DR$ operational cycle:

- **Remediate:** Instantaneous task shedding bounded by soft-overload fallbacks mitigates acute queue saturation during flash crowds and server dropouts.
- **Recover:** Brownout-aware load balancing coordinates workload re-admission, accelerating cluster return to healthy operational equilibria without re-inducing secondary queuing collapses.

In practical cloud-native production environments (such as Kubernetes clusters fronted by Envoy or Istio service meshes), CloudRescue offers significant operational advantages:

- **Bridging the Provisioning Delay Gap:** Horizontal Pod Autoscalers (HPA) and cloud VM provisioning require between 30 seconds and several minutes to detect sustained overload, schedule pods, pull container images, and initialize runtime environments [1, 5]. CloudRescue provides instantaneous, sub-second protection during this transient provisioning window, shedding optional tasks until reactive auto-scalers can instantiate additional physical instances.
- **Service Mesh Implementation:** BALB can be directly implemented as an Envoy HTTP filter using WebAssembly (WASM), evaluating lightweight dimmer headers returned in upstream response metadata without requiring out-of-band coordination.
- **Decomposition Guidelines:** Microservice developers should partition service contracts into mandatory transactional paths (e.g., checkout, order persistence) and optional enhancement paths (e.g., personalized recommendations, speculative pre-computations), exposing dimmer probabilities via local middleware decorators.

G. Threats to Validity and Scientific Limitations

To maintain the highest standards of scientific integrity, we explicitly document the limitations and threats to validity inherent in this study:

1. **Simulation-Based Scope vs. Production Cloud Environments (External Validity):**
   This evaluation was conducted entirely within a discrete-event simulation model (SimPy [9]) with microsecond event scheduling. While discrete-event simulation provides complete determinism, perfect reproducible stream isolation, and eliminates uncontrolled background noise (such as OS scheduling interrupts, hypervisor CPU overcommitment, and garbage collection pauses), it does not capture complex physical-layer phenomena such as TCP connection window dynamics, network switch buffer bloat, hardware memory bus contention, or asynchronous distributed clock drift. Consequently, while relative algorithmic trade-offs are rigorously established, absolute latency values must be validated in production testbeds prior to enterprise deployment.
2. **Workload and Execution Time Distributions (Construct Validity):**
   Request inter-arrival intervals and component execution durations were modeled using standard Markovian exponential distributions. In production cloud microservices, execution times frequently follow heavy-tailed distributions (such as Pareto or log-normal distributions) where occasional large requests can cause head-of-line blocking. The impact of heavy-tailed service times on PI controller stability and BALB queue penalties represents an important open research question.
3. **Cumulative Utilization Inertia (Internal Validity):**
   In BALB, worker utilization is defined as cumulative processor busy time divided by operational uptime ($T_{\text{busy},w}/T_{\text{operational},w}$). While cumulative utilization provides exceptional long-term stability, it responds sluggishly to sudden, short-duration workload changes after extended periods of nominal operation. In production environments, an Exponential Moving Average (EMA) or sliding-window utilization metric may provide faster responsiveness to abrupt traffic shifts.
4. **Deterministic Join-Shortest-Queue Baseline (Internal Validity):**
   The baseline JSQ routing policy evaluated in `CFG-A` and `CFG-B` employs deterministic tie-breaking (selecting the lowest worker index on queue depth ties). While this reflects standard FIFO proxy implementations, an auxiliary comparative baseline evaluating randomized tie-breaking (Randomized JSQ / JSQ-R) was not implemented in this experimental battery.
5. **Exploratory Sensitivity Sample Size (Statistical Conclusion Validity):**
   The multi-factor parameter sweep across BALB routing weights evaluated 48 combinations over $n = 3$ seeds per cell (144 runs). While this sweep confirms the broad stability and boundaries of the parameter landscape, three seeds per

cell provide insufficient statistical power to identify a single globally optimal mathematical coordinate. Top-ranking parameter configurations are reported strictly as exploratory observations.

6. **Right-Censoring Horizon Substitution (Statistical Conclusion Validity):**
In our post-restoration recovery evaluation, substituting the 550.0-second observation cutoff for unrecovered runs represents a heuristic single-imputation approach. It does not constitute formal non-parametric survival analysis (such as the Kaplan-Meier product-limit estimator [17] or semi-parametric Cox proportional hazards modeling). The observed speedup ratios ($2.06\times$ to $2.15\times$) are descriptive findings under the fixed horizon and should not be interpreted as mathematically proven theoretical lower bounds.
7. **Lower Service Fidelity in BALB (Construct Validity):**
As candidly reported, full BALB (`CFG-C`) consistently delivers lower service fidelity ($F = 9.5\%$–13.8%) than uncoordinated brownout (`CFG-B` , $F = 20.9\%$–29.7%). Service providers whose business models heavily prioritize optional feature execution over workload distribution equity may find BALB's aggressive degradation counter-productive unless weighting parameters (α, β, γ) are tuned specifically to favor fidelity.
8. **Single-Tier Cluster Scope (External Validity):**
Our modeled architecture comprises a single centralized ingress proxy fronting a single-tier worker cluster ($M = 4$). Modern enterprise cloud applications typically consist of deep, multi-tier microservice call graphs with cascading dependencies. Recursive brownout propagation across multi-tier topologies remains an open avenue for future investigation.

IX. CONCLUSION AND FUTURE WORK

A. Summary of Contributions

In this paper, we presented **CloudRescue**, an autonomic cloud resilience framework that unites local closed-loop brownout feedback control with brownout-aware multi-attribute request dispatching to protect cloud microservices from catastrophic queuing degradation during severe operational shocks. Modern cloud systems face acute vulnerability to traffic flash crowds and hardware dropouts; traditional mitigation via hardware over-provisioning is economically prohibitive, while reactive auto-scaling suffers from non-negligible provisioning delays that leave systems unprotected during the onset of disruptions [1, 5].

To address these limitations, CloudRescue introduced three core architectural and methodological contributions:

1. **Velocity-Form Discrete PI Brownout Controller with Intrinsic Anti-Windup:** We formulated an autonomous, per-worker feedback controller that modulates the execution probability of optional software features ($\theta_w \in [0, 1]$). By computing control candidates directly from clamped outputs, incorporating an insufficient-sample hold mechanism, and suppressing derivative kicks during telemetry resumption, our controller provides stable, sub-second capacity modulation without integral windup lag.
2. **Brownout-Aware Load Balancing (BALB):** We formulated a multi-attribute dispatch scoring algorithm ($Score_w = \alpha\theta_w - \beta\frac{q_w}{Q_{max}} - \gamma U_w$) that decouples dispatching from raw queue depth, incorporates dynamic queue back-off penalties, and enforces an autonomic soft-overload fallback. BALB resolves the pathological load skew of Join-Shortest-Queue under brownout, restoring near-perfect load balance across active workers ($J \geq 0.981$–0.996).
3. **Rigorous Three-Way Ablation and Reproducible Discrete-Event Framework:** Evaluated across 360 primary simulation runs with 5-stream PRNG Common Random Numbers stream isolation and a 144-run exploratory sensitivity sweep, our study cleanly isolated baseline operation (`CFG-A`), local brownout alone (`CFG-B`), and full BALB (`CFG-C`). We introduced the DEC-010 nominal-referenced recovery methodology to eliminate artificial right-censoring in post-repair evaluation, establishing that BALB accelerates post-restoration recovery by $2.06\times$ to $2.15\times$ ($p < 4.1 \times 10^{-5}$). Crucially, our empirical analysis characterized the fundamental trade-off between workload distribution equity and optional service fidelity, showing that BALB intentionally sacrifices feature richness (F) by 11.4 to 15.9 percentage points ($p < 10^{-38}$) to maintain cluster-wide queue stability and SLA compliance.

B. Future Research Directions

The findings and limitations documented in this research highlight several promising avenues for future investigation:

1. **Multi-Tier Microservice Dependency Topologies:** While CloudRescue modeled an ingress proxy fronting a single-tier worker pool, modern enterprise applications consist of complex directed acyclic graph (DAG) call trees. Extending brownout control across multi-tier service graphs requires investigating recursive dimmer propagation, distributed backpressure coordination, and multi-tier BALB dispatching.
2. **Trace-Driven and Heavy-Tailed Workload Evaluation:** Future work will evaluate CloudRescue under production cloud traces (e.g., Azure Functions or Alibaba cluster traces) characterized by heavy-tailed service times (Pareto, log-normal) and diurnal arrival patterns, assessing controller convergence and queue penalty dynamics under non-exponential execution profiles.
3. **Online Adaptive Tuning via Machine Learning:** While our 144-run sensitivity sweep confirmed the broad stability of default weights ($\alpha = 1.0, \beta = 1.0, \gamma = 0.5$), online reinforcement learning or Bayesian optimization could dynamically tune routing weights in response to changing traffic burstiness and failure severities.
4. **Physical Cloud Testbed and Service Mesh Deployment:** A natural extension is implementing CloudRescue within production-grade service mesh architectures (e.g., Envoy proxy sidecars using WebAssembly filters) deployed on Kubernetes, evaluating real-world interactions with Linux kernel networking, container runtime overheads, and horizontal pod autoscalers.

REFERENCES

[1] T. Welsh and E. Benkhelifa, "On Resilience in Cloud Computing: A Survey of Techniques across the Cloud Domain," *ACM Computing Surveys*, vol. 53, no. 3, art. 59, pp. 59:1–59:36, May 2020. DOI: 10.1145/3388922.

[2] C. Klein, A. V. Papadopoulos, M. Manfredi, I. Epifani, A. Leva, and K.-E. Årzén, "Brownout: Building More Robust Cloud Applications," in *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*, Hyderabad, India: ACM, 2014, pp. 700–711. DOI: 10.1145/2568225.2568227.

[3] C. Klein, M. Maggio, K.-E. Årzén, and F. Hernández-Rodriguez, "Improving Cloud Service Resilience Using Brownout-Aware Load-Balancing," in *Proceedings of the 2014 IEEE 33rd International Symposium on Reliable Distributed Systems (SRDS '14)*, Nara, Japan: IEEE Computer Society, 2014, pp. 31–40. DOI: 10.1109/SRDS.2014.14.

[4] J. P. G. Sterbenz, D. Hutchison, E. K. Çetinkaya, A. Jabbar, J. P. Rohrer, M. Schöller, and P. Smith, "Resilience and survivability in communication networks: Strategies, principles, and survey of disciplines," *Computer Networks*, vol. 54, no. 8, pp. 1245–1265, Jun. 2010. DOI: 10.1016/j.comnet.2010.03.005.

[5] J. Cartlidge and I. Sriram, "Modelling resilience in cloud-scale data centres," in *Proceedings of the 23rd European Modeling & Simulation Symposium (EMSS '11)*, Rome, Italy, Sep. 2011, pp. 1–10. [Online]. Available: arXiv:1106.5457.

[6] J.-C. Laprie, "From dependability to resilience," in *Proceedings of the 38th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '08)*, Anchorage, AK, USA: IEEE, 2008, pp. G8–G9. DOI: 10.1109/DSN.2008.4630069.

[7] Z. Zheng, T. C. Zhou, M. R. Lyu, and I. King, "FTCloud: A Component Ranking Framework for Fault-Tolerant Cloud Applications," in *Proceedings of the 2010 IEEE 21st International Symposium on Software Reliability Engineering (ISSRE '10)*, San Jose, CA, USA: IEEE, 2010, pp. 398–407. DOI: 10.1109/ISSRE.2010.28.

[8] L. Tomas and J. Tordsson, "An Autonomic Approach to Risk-Aware Overbooking in Cloud Computing," *IEEE Transactions on Cloud Computing*, vol. 2, no. 3, pp. 292–305, Jul.–Sep. 2014. DOI: 10.1109/TCC.2014.2326165.

[9] Team SimPy, "SimPy: Discrete-event simulation for Python," Python Software Foundation, PyPI package version 4.x, 2024. [Online]. Available: https://simpy.readthedocs.io/.

[10] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. Boston, MA, USA: McGraw-Hill, 2015. ISBN: 978-0073401324.

[11] R. Jain, D.-M. W. Chiu, and W. R. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," Digital Equipment Corporation, Hudson, MA, USA, Technical Report DEC-TR-301, Sep. 1984.

[12] S. S. Shapiro and M. B. Wilk, "An analysis of variance test for normality (complete samples)," *Biometrika*, vol. 52, no. 3–4, pp. 591–611, Dec. 1965. DOI: 10.1093/biomet/52.3-4.591.

[13] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, Dec. 1945. DOI: 10.2307/3001968.

[14] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ, USA: Lawrence Erlbaum Associates, 1988. ISBN: 978-0805802832.

[15] C. E. Bonferroni, "Teoria statistica delle classi e calcolo delle probabilità," *Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze*, vol. 8, pp. 3–62, 1936.

[16] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995. DOI: 10.1111/j.2517-6161.1995.tb02031.x.

[17] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," *Journal of the American Statistical Association*, vol. 53, no. 282, pp. 457–481, Jun. 1958. DOI: 10.1080/01621459.1958.10501452.

APPENDIX: REPOSITORY ARTIFACTS & VERIFICATION PROVENANCE

The architectural specifications, experimental parameters, literature mappings, and empirical results presented in this paper are traceably derived from the certified CloudRescue research repository:

- **Foundational Literature:** Welsh and Benkhelifa [1], Klein et al. [2, 3], and Sterbenz et al. [4].
- **System Specifications:** Product Requirements Document (`CloudRescue_Project_Docs/PRD.md`) and System Architecture (`CloudRescue_Project_Docs/ARCHITECTURE.md`).
- **Methodological Decisions:** Formal Decision Log (`CloudRescue_Project_Docs/DECISIONS.md` , specifically DEC-007, DEC-008, DEC-009, and DEC-010).
- **Authoritative Experimental Data:** Master Dataset (`results/experiment_360/master_results.csv`), Aggregated Metrics (`results/experiment_360/aggregated_metrics.csv`), Statistical Battery (`results/experiment_360/statistical_comparisons.csv`), and Sensitivity Sweep (`results/experiment_360/sensitivity/balb_weights_aggregated.csv`).
- **Certified Freeze Signoff:** Final Research Freeze Report (`results/experiment_360/FINAL_RESEARCH_FREEZE_REPORT.md`) and Freeze Signoff (`FINAL_FREEZE_SIGNOFF.md`).
- **Audit Verification:** Section 8 Comprehensive Verification & Data Audit Report (`paper/SECTION_8_AUDIT_REPORT.md`) and Reference Verification Inventory (`paper/REFERENCES_TO_VERIFY.md`).